TRAVAUX PRATIQUES– FILIÈRE INTELLIGENCE ARTIFICIELLE

M108 - Appréhender l’apprentissage profond

40 heures

SOMMAIRE

01 - COMPRENDRE LES FONDAMENTAUX DU DEEP LEARNING

Activité 1 : Introduire les réseaux de neurones
Activité 2 : Maîtriser les techniques de régularisation

02 – MANIPULER LES RÉSEAUX DE NEURONES AVANCÉS

Activité 1 : Construire et optimiser des réseaux à convolution

03 - EXPLORER LES RÉSEAUX RÉCURRENTS

Activité 1 : Comprendre les réseaux récurrents et appliquer LSTM et GRU

MODALITÉS PÉDAGOGIQUES

1

LE GUIDE DE SOUTIEN

Il contient le résumé théorique et le manuel des travaux pratiques

2

LA VERSION PDF

Une version PDF est mise en ligne sur l'espace apprenant et formateur de la plateforme WebForce Life

3

DES CONTENUS TÉLÉCHARGEABLES

Les fiches de résumés ou des exercices sont téléchargeables sur WebForce Life

4

DU CONTENU INTERACTIF

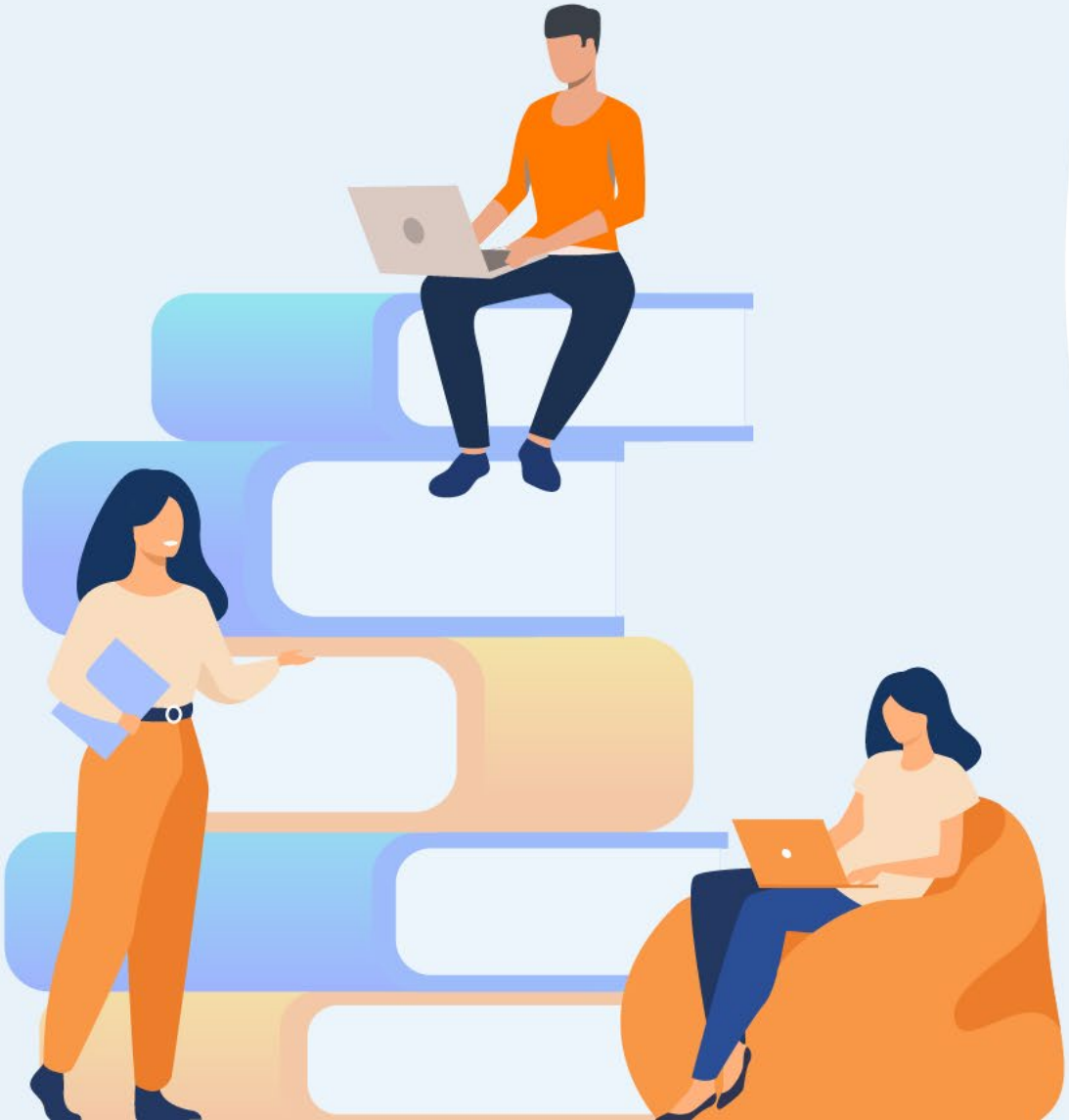
Vous disposez de contenus interactifs sous forme d'exercices et de cours à utiliser sur WebForce Life

5

DES RESSOURCES EN LIGNES

Les ressources sont consultables en synchrone et en asynchrone pour s'adapter au rythme de l'apprentissage

Copyright - Tout droit réservé - OFPPT



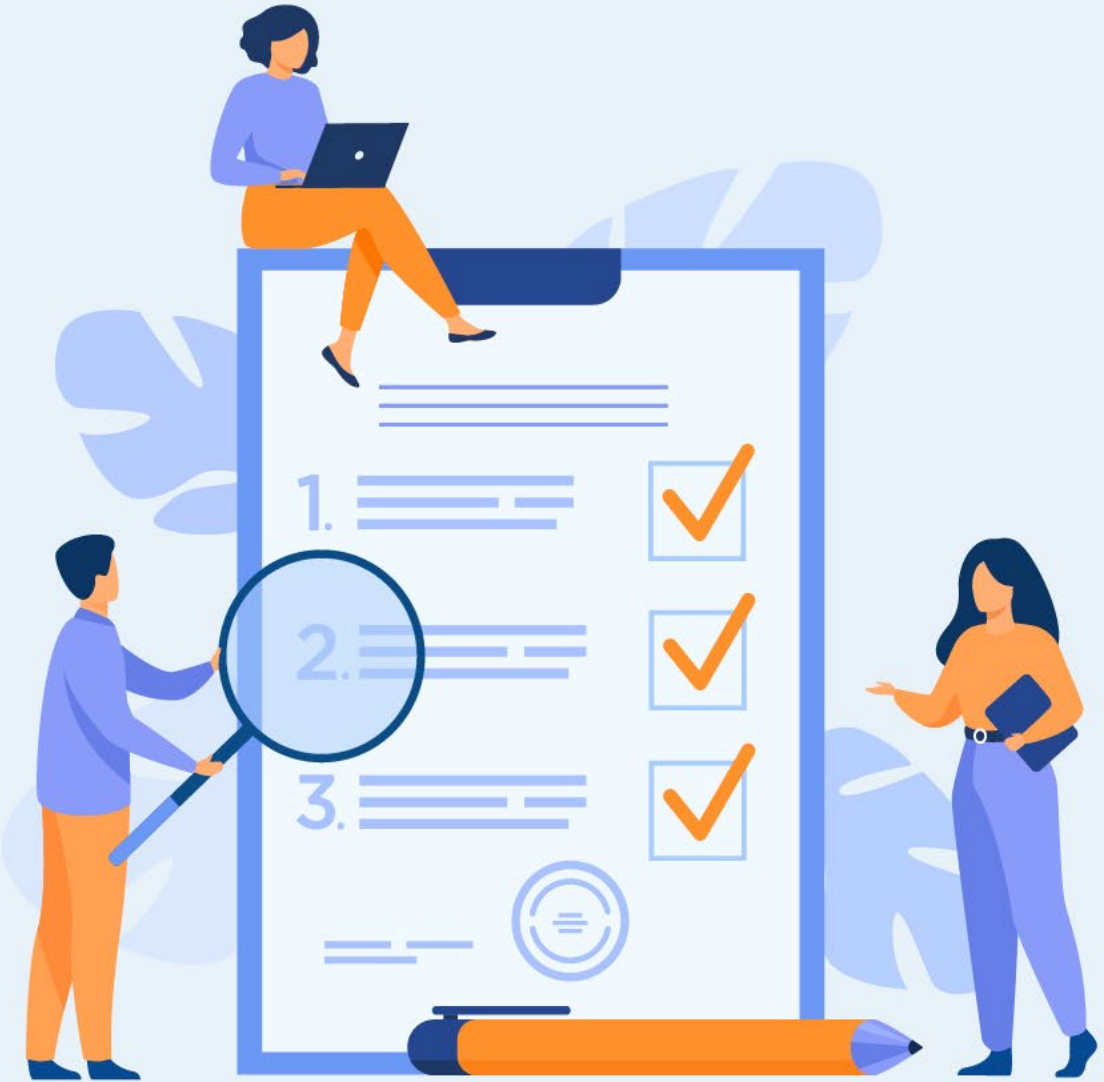
PARTIE 1

COMPRENDRE LES FONDAMENTAUX DU DEEP LEARNING

Dans ce module, vous allez :

- Introduire les réseaux de neurones
- Maîtriser les techniques de régularisation

10 heures



Activité 1
INTRODUIRE LES RÉSEAUX DE NEURONES

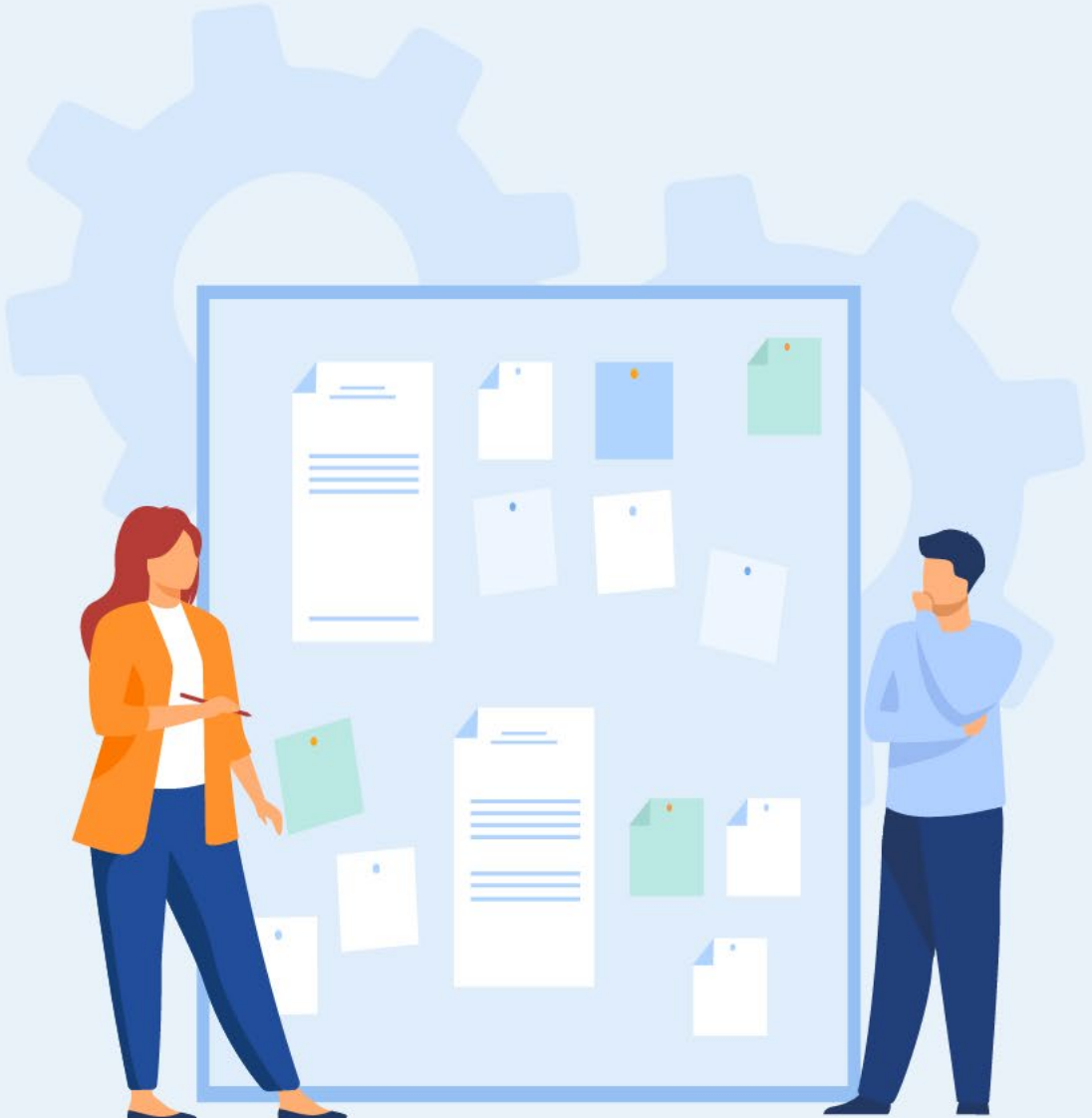
Compétences visées :

- Séparer une base de données en données d'entraînement (training) et de test (testing)
- Assurer le prétraitement des données
- Définir un perceptron multicouche sous Keras et initialiser les paramètres
- Entraîner et évaluer le modèle

Recommandations clés :

- Faire une bonne lecture de l'énoncé de l'exercice
- Utiliser la bibliothèque Keras pour implémenter les réseaux de neurones

05 heures



CONSIGNES

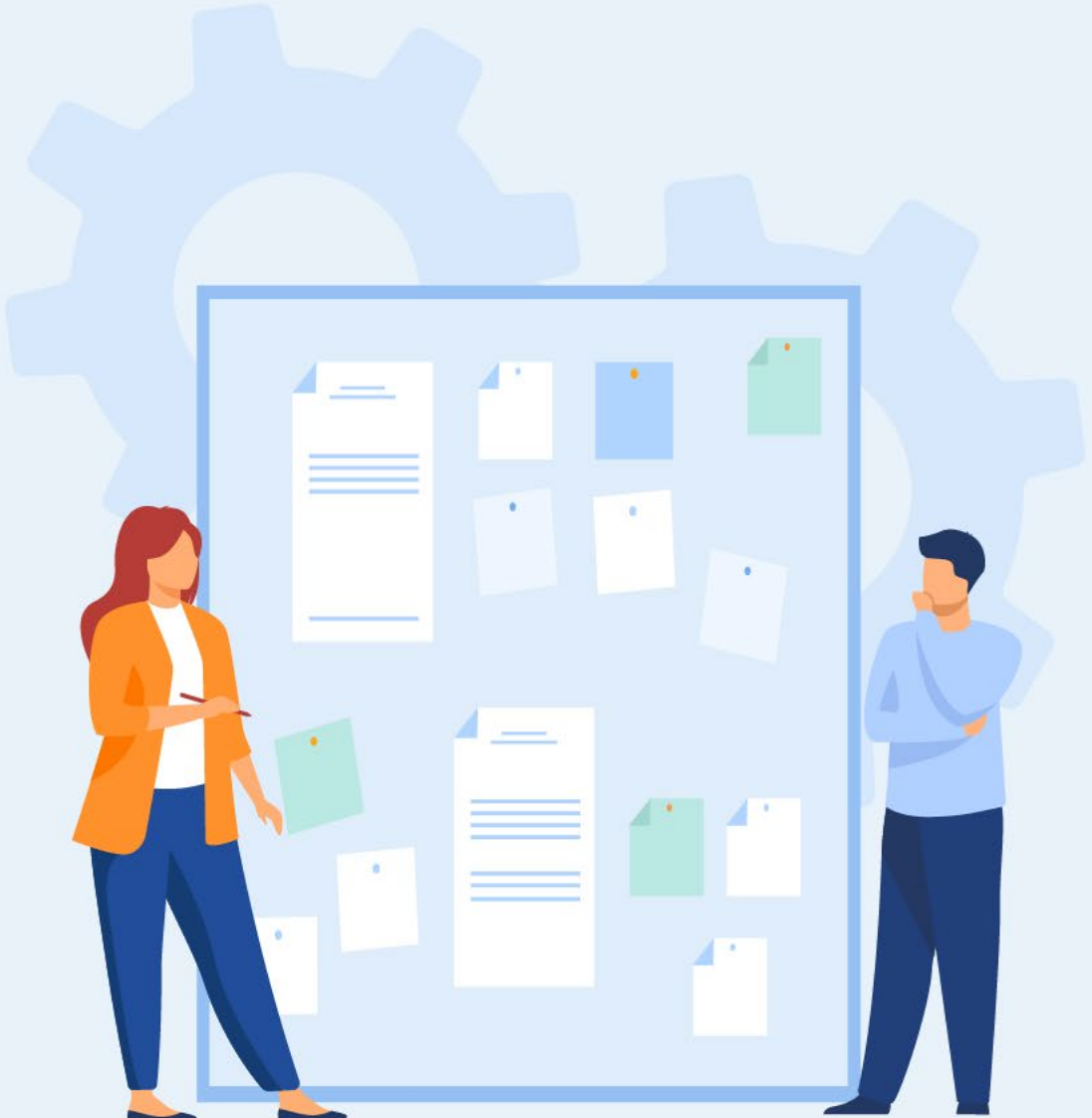
1- Pour le formateur

- Demander de charger et séparer la base de données
- Demander de préparer les données
- Demander de construire le réseau
- Demander d’entraîner et d’évaluer le réseau

2- Pour l’apprenant

- charger et séparer la base de données
- Préparer les données
- Construire le réseau
- Entraîner et évaluer le réseau

CONSIGNES



3 - Conditions de réalisation

- Support de résumé théorique accompagnant
- Suivi des instructions verbales ou écrites du formateur
- Travail individuel ou en groupe
- Utilisation d’un ordinateur
- Utilisation du Google Colab
- Acquisition d’une connexion internet

4 - Critères de réussite

Le stagiaire est-il capable de :

- Charger, séparer et préparer les données ?
- Entraîner et évaluer le modèle ?

Activité 1

Exercices

Exercice 1 : Classification des digits avec ANN

La base de données MNIST de chiffres manuscrits, comprend un ensemble d'entraînement de 60 000 exemples et un ensemble de test de 10 000 exemples. La première étape consiste à charger et à séparer la base de données MNIST qui existe dans le module **keras.datasets** en utilisant la fonction mnist.load_data().

```
import keras
from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

En Google Colab, ouvrir un nouveau notebook, copier le bout du code et répondre aux questions :

1. Quel est le shape des données ?
2. Quel est le nombre d'images des deux ensembles d'apprentissage et de test ?

La fonction **mnist.load_data()** génère deux tuples :

- ✓ **x_train, x_test**: un tableau des uint8 qui représentent des images en niveaux de gris avec le shape (num_samples, 28, 28) ;
- ✓ **y_train, y_test**: un tableau des uint8 qui représentent les étiquettes de chiffres (entiers compris entre 0 et 9) avec le shape (num_samples,).

Afin d'entraîner le perceptron multicouche (MLP) à classer les images de digits, on doit tout d'abord redimensionner les données du shape (28,28) en un seul vecteur 784 = 28 × 28 (le vecteur d'entrée du MLP). On va représenter graphiquement la distribution des valeurs de pixels pour un exemple quelconque. Interpréter l'histogramme du digit.

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1 : Classification des digits avec ANN

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
fig = plt.figure()
plt.subplot(2,1,1)
plt.imshow(x_train[0], cmap='gray', interpolation='none')
plt.title("Digit: {}".format(y_train[0]))
plt.xticks([])
plt.yticks([])
plt.subplot(2,1,2)
plt.hist(x_train[0].reshape(784))
plt.title("Pixel Value Distribution")
```

La normalisation des données d'entrée permet d'accélérer l'entraînement des réseaux de neurones. En outre, cela réduit les chances de rester coincé dans les optima locaux, car on utilise la descente de gradient stochastique pour trouver les poids optimaux. Pour cela, on va redimensionner nos entrées en un seul vecteur. Ensuite, on va normaliser les valeurs des pixels pour qu'elles se situent entre 0 et 1.

3. Implémenter le code afin de redimensionner et normaliser les données.
4. Afficher à nouveau les nouvelles valeurs d'un échantillon de l'ensemble d'apprentissage.

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1 : Classification des digits avec ANN

La plupart des algorithmes d'apprentissage automatique existants ne peuvent pas être exécutés sur des données catégorielles. Au lieu de cela, les données catégorielles doivent d'abord être converties en données numériques. Le codage « one-hot » est l'une des techniques utilisées pour effectuer cette conversion.

Cette méthode est principalement utilisée lorsque des techniques d'apprentissage profond doivent être appliquées à des problèmes de classification séquentielle. Le codage « one-hot » est essentiellement la représentation de variables catégorielles sous forme de vecteurs binaires. Ces valeurs catégorielles sont d'abord mappées sur des valeurs entières. Chaque valeur entière est alors représentée comme un vecteur binaire qui est rempli de 0 (sauf l'index de l'entier qui est marqué comme 1). Dans notre cas, le résultat pour chaque donnée est un vecteur d'une longueur égale au nombre de catégories (10=chiffres de 0 à 9) qui est rempli de neuf zéros et un dans la position de la catégorie du digit en question. Ainsi, le digit « 5 » sera représenté par [0,0,0,0,1,0,0,0,0].

5. En utilisant **np_utils.to_categorical** de Keras, appliquer le one-hot encoding sur les labels d'apprentissage et de test.

Les modèles dans Keras sont définis comme une séquence de couches. On crée un modèle séquentiel à l'aide de la fonction **Sequential()**. On empile les différentes couches en utilisant la fonction **add()**. D'après la Figure 1, notre vecteur de 784 pixels représente l'entrée du réseau. Ensuite, la couche cachée contient 512 nœuds pour apprendre la dépendance entre les données d'entrée. Pour la classification multi-classes, on ajoute une autre couche entièrement connectée pour les 10 classes de sortie. Lors de l'ajout de la première couche dans le modèle séquentiel, on doit spécifier la forme d'entrée input_shape=(784,) afin que Keras puisse créer les matrices appropriées. Pour toutes les couches restantes, la forme est automatiquement déduite. Les couches entièrement connectées sont définies à l'aide de la classe **Dense**. On peut spécifier le nombre de neurones ou de nœuds dans la couche comme premier argument, et spécifier la fonction d'activation en utilisant l'argument d'activation. On utilise la fonction d'activation ReLU (Rectified Linear Unit) sur la couche cachée et la fonction softmax dans la couche de sortie.

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1 : Classification des digits avec ANN

De nos jours, de meilleures performances sont obtenues en utilisant la fonction d'activation ReLU. On utilise un softmax sur la couche de sortie pour s'assurer que la sortie de notre réseau est comprise entre 0 et 1. La dernière couche contient les connexions pour nos 10 classes et la fonction d'activation softmax qui est standard pour la classification multi-classes.

6. Implémenter le modèle en se basant sur la **Figure 1**.

Maintenant que le modèle est mis en place, on configure le processus d'entraînement à l'aide de la fonction **compile().** Quant à l'optimiseur, on utilise SGD (Stochastic Gradient Descent) avec les paramètres suivants :

- ✓ learning rate=0.01
- ✓ momentum=0.9
- ✓ nesterov=True
- ✓ Batch_size=128

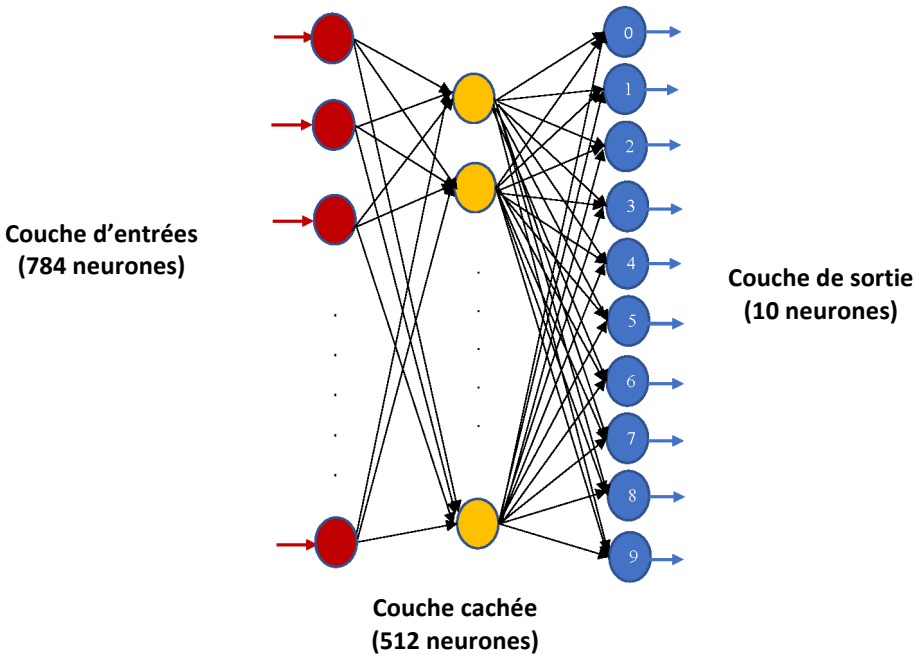


Figure 1: MLP avec une seule couche cachée.

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1

Une fonction de perte (fonction objective ou fonction de score d'optimisation) est l'un des trois paramètres requis pour compiler un modèle. On sélectionne « categorical_crossentropy » qui est utilisée pour la catégorisation d'une seule étiquette. Une seule catégorie est applicable pour chaque point de données. En d'autres termes, un exemple ne peut appartenir qu'à une seule classe. On choisit également les métriques qui seront évaluées lors de l'entraînement et des tests, à savoir la justesse (accuracy).

7. Implémenter le bout de code qui permet de compiler et d'entraîner le modèle pendant 20 époques.

Une fois notre modèle est entraîné, on va l'évaluer en utilisant la fonction **evaluate().**

8. Implémenter le code qui permet d'évaluer le modèle sur l'ensemble de test. Puis afficher la valeur de perte (loss) et de justesse (accuracy).

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

1. Quel est le shape des données ?

```
print(x_train.shape, 'Ensemble entraînement')
print(x_test.shape, 'Ensemble test')
```

```
(60000, 28, 28) Ensemble entraînement
(10000, 28, 28) Ensemble test
```

Le shape des données est (28,28).

2. Quel est le nombre d'images des deux ensembles d'apprentissage et de test ?

- ✓ Le nombre d'images de l'ensemble d'apprentissage est : 60 000
- ✓ Le nombre d'images de l'ensemble de test est : 10 000

3. Implémenter le code afin de redimensionner et normaliser les données :

```
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

4. Afficher à nouveau les nouvelles valeurs d'un échantillon de l'ensemble d'apprentissage :

```
print("Après normalisation", x_train[0])
```

```
Après normalisation [0.         0.         0.         0.
0.         0.
 0.         0.         0.         0.         0.         0.
 0.         0.         0 0.         0.         0.01176471
0.07058824 0.07058824 0.07058824
 0.49411765 0.53333336 0.6862745  0.10196079 0.6509804  1.
 0.96862745 0.49803922 0.         0.         0.         0.
 0.         0.         0.         0.         0.         0.
 0.         0.         0.11764706 0.14117648 0.36862746
0.6039216
 0.6666667  0.99215686 0.99215686 0.99215686 0.7764706
0.7137255
 0.96862745 0.94509804 0.         0.         0.         0.
 0.         0.         0.         0.         0.         0.
 0.         0.         0.         0.         0.         0.
0.         0.         0.1764706  0.7294118  0.99215686
0.99215686
 0.5882353  0.10588235 0.         0.         0.         0.
 0.         0.         0.         0.         0.         0.
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

5. En utilisant np_utils.to_categorical de Keras, appliquer le one-hot encoding sur les labels d'apprentissage et de test :

```
from keras.utils import np_utils
num_classes=10
print("Shape avant one-hot encoding: ", y_train.shape)
y_train = keras.utils.np_utils.to_categorical(y_train,
num_classes)
y_test = keras.utils.np_utils.to_categorical(y_test,
num_classes)
print("Shape après one-hot encoding: ", y_train.shape)
```

```
Shape avant one-hot encoding:  (60000,)
Shape après one-hot encoding:  (60000, 10)
```

6. Implémenter le modèle en se basant sur la Figure 1 :

```
from keras.models import Sequential
from keras.layers import Dense, Dropout
model = Sequential()
model.add(Dense(512, activation='relu',
input_shape=(784,)))
model.add(Dense(num_classes, activation='softmax'))
model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_2 (Dense)             (None, 512)               401920

 dense_3 (Dense)             (None, 10)                5130
=================================================================
Total params: 407,050
Trainable params: 407,050
Non-trainable params: 0
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

7. Implémenter le bout de code qui permet de compiler et d'entraîner le modèle pendant 20 époques :

```
batch_size = 128
epochs = 20
from tensorflow.keras import optimizers
sgd = optimizers.SGD(learning_rate=0.01, momentum=0.9,
nesterov=True)

model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                    batch_size=batch_size,
                    epochs=epochs,
                    verbose=1,
                    validation_data=(x_test, y_test))
```

```
Epoch 1/20
469/469 [==============================] - 5s
9ms/step - loss: 1.1065 - accuracy: 0.7509 -
val_loss: 0.6016 - val_accuracy: 0.8651
Epoch 2/20
469/469 [==============================] - 4s
9ms/step - loss: 0.5240 - accuracy: 0.8704 -
val_loss: 0.4342 - val_accuracy: 0.8906
Epoch 3/20
469/469 [==============================] - 5s
10ms/step - loss: 0.4217 - accuracy: 0.8895 -
val_loss: 0.3740 - val_accuracy: 0.9029
Epoch 4/20
469/469 [==============================] - 4s
8ms/step - loss: 0.3750 - accuracy: 0.8992 -
val_loss: 0.3411 - val_accuracy: 0.9084
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

8. Implémenter le bout de code qui permet d'évaluer le modèle sur l'ensemble de test. Puis afficher la valeur de perte (loss) et de justesse (accuracy) :

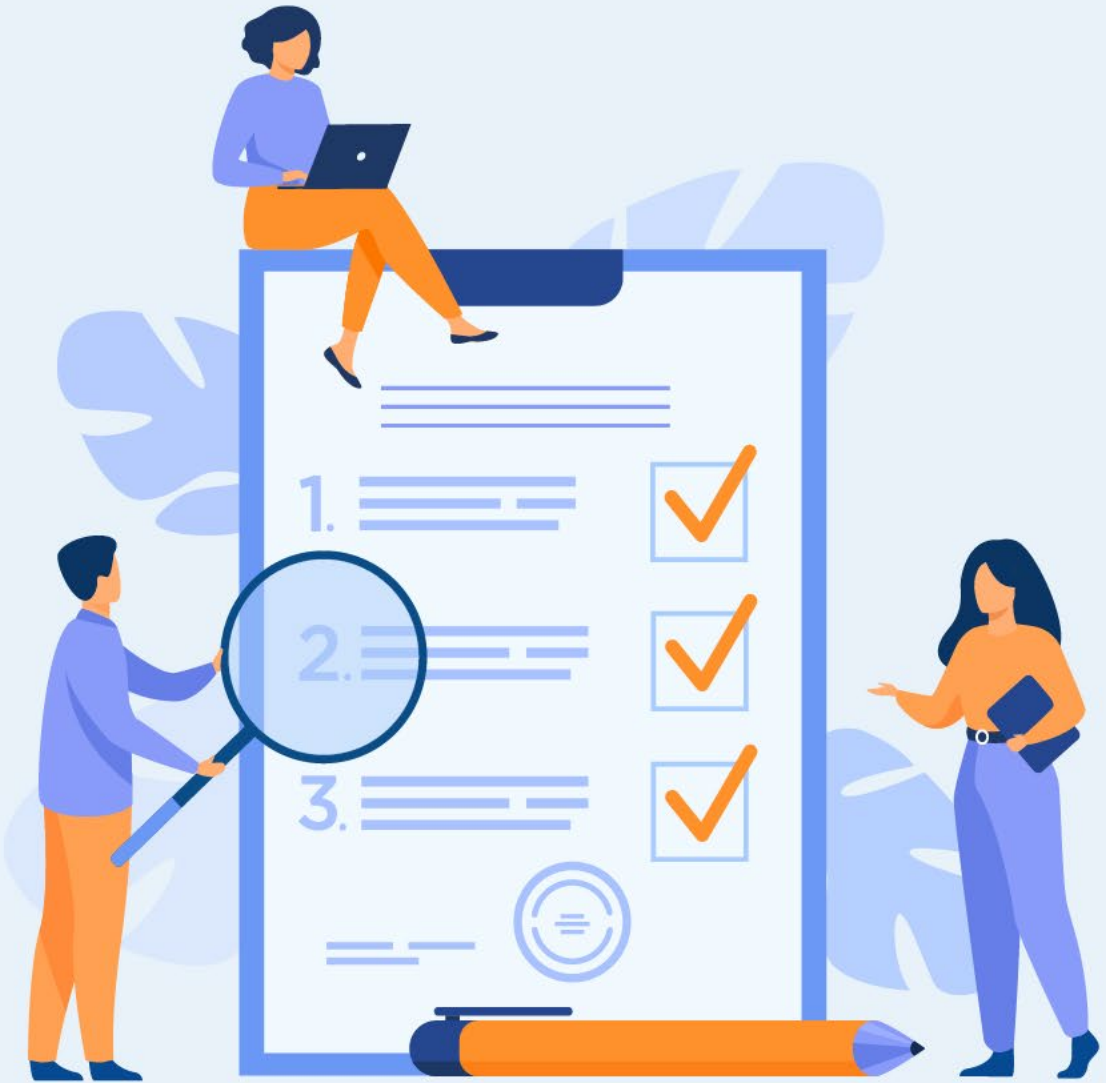
```
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

```
Test loss: 0.20572878420352936
Test accuracy: 0.9423999786376953
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2
MAÎTRISER LES TECHNIQUES DE RÉGULARISATION



Compétences visées :

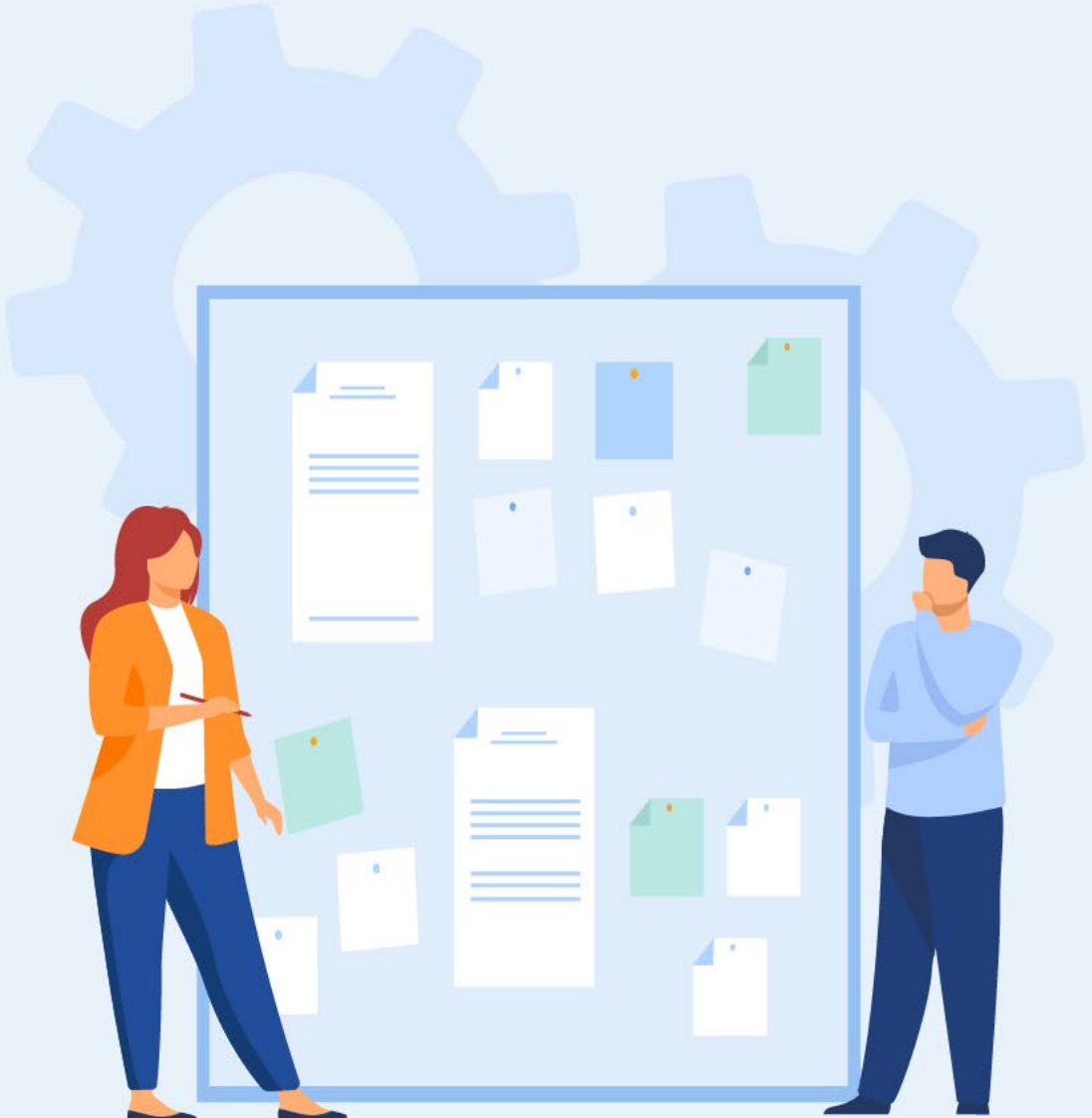
- Implémenter différentes techniques de régularisation
- Classer les sentiments en se basant sur les techniques du NLTK

Recommandations clés :

- Utiliser la bibliothèque Keras pour implémenter les techniques de régularisation

05 heures

CONSIGNES



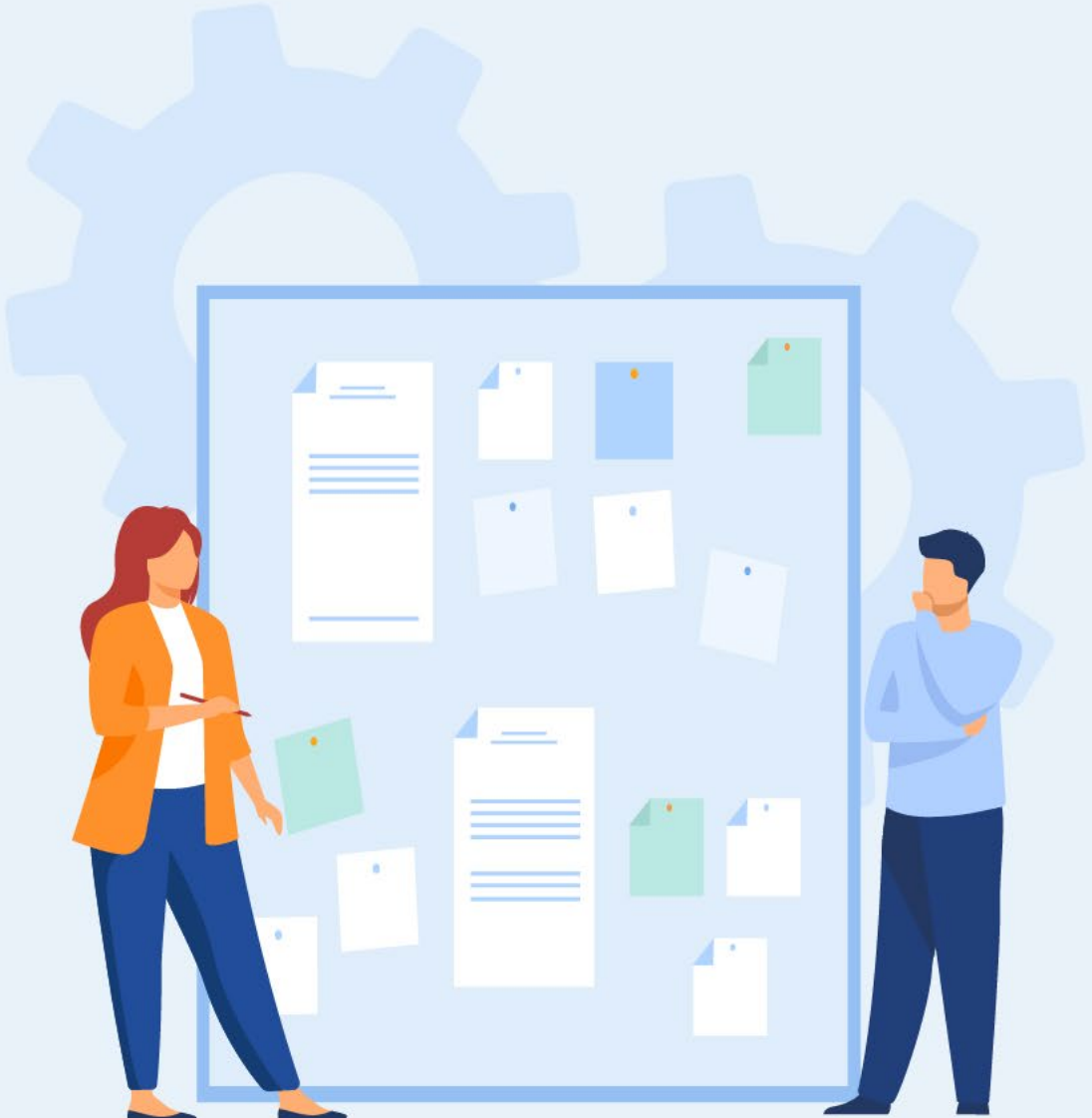
1- Pour le formateur

- Demander d’installer NLTK
- Demander de préparer les données
- Demander de créer un réseau de neurones
- Demander d’entraîner et d’évaluer ce réseau
- Demander de gérer le surapprentissage

2- Pour l’apprenant

- Installer NLTK
- Préparer les données
- Créer un réseau de neurones
- Entraîner et d’évaluer ce réseau
- Gérer le surapprentissage

CONSIGNES



3 - Conditions de réalisation

- Support de résumé théorique accompagnant
- Suivi des instructions verbales ou écrites du formateur
- Travail individuel ou en groupe
- Utilisation d’un ordinateur
- Utilisation de Google Colab
- Acquisition d’une connexion internet

4 - Critères de réussite

- Le stagiaire est-il capable de :
  - Préparer les données ?
  - Utiliser NLTK ?
  - Gérer le surapprentissage ?

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

Phase 1: Installation

Natural Language Toolkit (NLTK) (https://www.nltk.org/) est une boîte-à-outil permettant la création de programmes pour l'analyse de texte. Cet ensemble a été créé à l'origine par Steven Bird et Edward Loper à l'Université de Pennsylvanie en 2001. Pour installer le NLTK, taper :

```
pip install nltk
```

NLTK est livré avec de nombreux corpus, grammaires, modèles pré-entraînés, etc. Une liste complète est publiée sur http://www.nltk.org/nltk_data/. Pour installer les données, taper la commande suivante :

```
import nltk
nltk.download('stopwords')
```

Afin d'implémenter le modèle de classification des sentiments, on aura besoin de ces paramètres :

```
NB_WORDS = 10000          # Nombre de mots du dictionnaire
NB_START_EPOCHS = 20      # Nombre d'époques
BATCH_SIZE = 512          # Taille du lot
MAX_LEN = 20              # Maximum de mot dans une séquence
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

Phase 2: Fonctions utiles

deep_model() : Fonction pour entraîner un modèle multi-classe. Le nombre d'époques et batch_size sont définies par des valeurs dans la partie déclaration et initialisation.

Paramètres :

- ✓ model : modèle avec l'architecture choisie
- ✓ X_train : données d'entraînement
- ✓ y_train : étiquettes d'entraînement
- ✓ X_valid : données de validation
- ✓ Y_valid : étiquettes de validation

Output : Historique de l'entraînement du modèle

```
def deep_model(model, X_train, y_train, X_valid, y_valid):
    model.compile(optimizer='rmsprop'
                  , loss='categorical_crossentropy'
                  , metrics=['accuracy'])

    history = model.fit(X_train
                        , y_train
                        , epochs=NB_START_EPOCHS
                        , batch_size=BATCH_SIZE
                        , validation_data=(X_valid, y_valid)
                        , verbose=0)
    return historyg
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

eval_metric() : Fonction permettant d'évaluer un modèle entraîné sur une métrique choisie. La métrique d'entraînement et de validation est tracée dans un graphique linéaire pour chaque époque.

Paramètres :

- ✓ history: historique de l'entraînement du modèle
- ✓ metric_name : perte (loss) ou justesse (accuracy)

Output : Un graphique avec les époques dans l'axe des x et la métrique dans l'axe y

```
import matplotlib.pyplot as plt
def eval_metric(model, history, metric_name):

    metric = history.history[metric_name]
    val_metric = history.history['val_' + metric_name]

    e = range(1, NB_START_EPOCHS + 1)

    plt.plot(e, metric, 'bo', label='Apprentissage ' + metric_name)
    plt.plot(e, val_metric, 'b', label='Validation ' + metric_name)
    plt.xlabel('Epoch number')
    plt.ylabel(metric_name)
    plt.title('Comparaison entre apprentissage et validation ' +
metric_name + ' for ' + model.name)
    plt.legend()
    plt.show()
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

test_model() : Fonction pour tester le modèle sur de nouvelles données après l'avoir entraîné sur l'ensemble des données d'entraînement avec le nombre optimal d'époques.

Paramètres :

- ✓ model : modèle entraîné
- ✓ x_train : données d'entraînement
- ✓ y_train : étiquettes d'entraînement
- ✓ x_test : données de test
- ✓ y_test : étiquettes de test
- ✓ Époques : nombre optimal d'époques

Output : Justesse et perte de test

```
def test_model(model, X_train, y_train, X_test,
y_test, epoch_stop):
    model.fit(X_train
              , y_train
              , epochs=epoch_stop
              , batch_size=BATCH_SIZE
              , verbose=0)
    results = model.evaluate(X_test, y_test)
    print()
    print('Test accuracy:
{0:.2f}%'.format(results[1]*100))
    return results
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

compare_models_by_metrics() : Fonction pour comparer une métrique entre deux modèles.

Paramètres :

- ✓ model_1 : modèle 1
- ✓ model_2 : modèle 2
- ✓ model_hist_1 : historique d'entraînement du modèle 1
- ✓ model_hist_2 : historique d'entraînement du modèle 2
- ✓ metrics : métrique à comparer, perte, acc, val_loss ou val_acc

Output : Graphique des métriques des deux modèles

```
def compare_models_by_metric(model_1, model_2,
model_hist_1, model_hist_2, metric):
    metric_model_1 = model_hist_1.history[metric]
    metric_model_2 = model_hist_2.history[metric]

    e = range(1, NB_START_EPOCHS + 1)
    metrics_dict = {
        'acc' : 'Accuracy Apprentissage',
        'loss' : 'Loss Apprentissage',
        'val_acc' : 'Accuracy Validaton',
        'val_loss' : 'Loss Validation'
    }
    metric_label = metrics_dict[metric]
    plt.plot(e, metric_model_1, 'ro', label=model_1.name)
    plt.plot(e, metric_model_2, 'r', label=model_2.name)
    plt.xlabel('Nombre Epoques')
    plt.ylabel(metric_label)
    plt.title('Comparaison de ' + metric_label + ' entre
les modèles')
    plt.legend()
    plt.show()
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1: Gérer le surapprentissage

optimal_epoch() : Fonction pour renvoyer le numéro d'époque où la perte de validation est minimale

Paramètres :

- ✓ model_hist : historique d'entraînement du modèle

Output : Numéro d'époque avec perte de validation minimale

```
def optimal_epoch(model_hist):
    min_epoch =
np.argmin(model_hist.history['val_loss']) + 1
    print("Perte de validation minimale atteinte à
l'époque {}".format(min_epoch))
    return min_epoch
```

remove_stopwords() : Fonction pour supprimer les stopwords anglais d'une série Pandas.

Paramètres :

- ✓ input_text : texte à nettoyer

Output : Série Pandas nettoyée

```
import nltk
from nltk.corpus import stopwords
def remove_stopwords(input_text):
    stopwords_list = stopwords.words('english')
    whitelist = ["n't", "not", "no"]
    words = input_text.split()
    clean_words = [word for word in words if (word
not in stopwords_list or word in whitelist) and
len(word) > 1]
    return " ".join(clean_words)
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

remove_mentions() : Fonction pour supprimer les mentions, précédées par @, dans une série Pandas

Paramètres :

- ✓ input_text: texte à nettoyer

Output : Série Pandas nettoyée

```
import re
def remove_mentions(input_text):
    return re.sub(r'@\w+', '', input_text)
```

Phase 3 : Préparation des données

1. Lire le fichier « Tweets.csv » et afficher son contenu.

Il est recommandé de mélanger les données avant de les séparer entre un ensemble d'entraînement et un ensemble de test. De cette façon, les classes de sentiment sont réparties également sur ces deux ensembles. Taper ce bout de code :

```
df = df.reindex(np.random.permutation(df.index))
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

2. Conserver que la colonne « text » comme entrée et la colonne « airlines_sentiment » comme cible.
3. Supprimer les stopwords et les mentions.
4. Séparer les données en 90% pour l'apprentissage et 10% restantes pour le test

Pour utiliser le texte comme entrée d'un modèle, on doit tout d'abord convertir les mots en tokens, ce qui signifie simplement convertir les mots en entier qui font référence à un index dans un dictionnaire. Ici, on ne conserve que les mots les plus fréquents dans l'ensemble d'apprentissage. On nettoie le texte en appliquant des filtres et en mettant les mots en minuscule. Les mots sont séparés par des espaces.

Pour cela, on va utiliser la classe Tokenizer() de Keras. Les arguments de Tokenizer() sont :

Arguments :

- ✓ **num_words:** le nombre maximum de mots à conserver, basé sur la fréquence des mots. Seuls les mots les plus courants seront conservés.
- ✓ **Filters:** une chaîne où chaque élément est un caractère qui sera filtré des textes. La valeur par défaut est toute la ponctuation, plus les tabulations et les sauts de ligne, moins le caractère '.

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

- ✓ **lower:** booléen. Indique s'il faut convertir les textes en minuscules.
- ✓ **split:** str. Séparateur pour la division des mots.
- ✓ **char_level:** si True, chaque caractère sera traité comme un token.
- ✓ **oov_token:** s'il est donné, il sera ajouté à **word_index** et utilisé pour remplacer les mots hors vocabulaire lors des appels **text_to_sequence**
- ✓ La méthode **fit_on_texts()** prend en argument les textes à traiter.

On peut afficher le nombre de textes, le nombre de mots et le comptage des mots en utilisant les attributs **document_count**, **num_words** et **words_counts**, respectivement.

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

```
from keras.preprocessing.text import Tokenizer
import collections
tk = Tokenizer(num_words=NB_WORDS,
               filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n',
               lower=True,
               char_level=False,
               split=' ')
tk.fit_on_texts(X_train)

print('Ajustement du tokenizer sur {} documents'.format(tk.document_count))
print('{} mots du dictionnaire'.format(tk.num_words))
print('Top 5 mots les plus courants :', collections.Counter(tk.word_counts).most_common(5))
print(tk.word_index)
```

5. En utilisant la méthode **texts_to_matrix** du **Tokenizer**, convertir le texte d'un tweet en un vecteur avec des valeurs **NB_WORDS**. Avec mode = binaire, il contient un indicateur indiquant si le mot est apparu dans le tweet ou non.

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Exercice 1 : Gérer le surapprentissage

6. Convertir les classes cibles en nombres en utilisant **LabelEncoder()** du Sklearn, puis appliquer le one-hot-encoding avec la méthode **to_categorical()** du Keras.

7. Séparer les données d'apprentissage en 90% de données d'apprentissage et 10% de données de validation. Cet ensemble de validation sera utilisé pour évaluer les performances du modèle.

Phase 4 : Modèle

Notre modèle contient deux couches densément connectées de 64 nœuds. Le input_shape pour la première couche est égale au nombre de mots que nous avons conservés dans le dictionnaire et pour lesquels nous avons créé des caractéristiques qui sont one-hot encodées.

Comme nous devons prévoir trois classes de sentiments différentes, la dernière couche a trois éléments. La fonction d'activation softmax garantit que les trois probabilités sont égales à 1.

Le nombre de paramètres à entraîner est calculé comme **((nb_entrées+1) x nb** éléments dans la couche cachée). Le nombre d'entrées pour la première couche est égal au nombre de mots dans notre dictionnaire. Les couches suivantes ont le nombre de sorties de la couche précédente comme entrées.

Copyright - Tout droit réservé - OFPPT

Activité 2

Exercices

Exercice 1 : Gérer le surapprentissage

Le nombre de paramètres par couche est donc :

- ✓ **Première couche**: (10000+1 x 64)= 640064
- ✓ **Deuxième couche**: (64+1 x 64) + 64 = 4160
- ✓ **Dernière couche:** (64+1 x 3) + 3 = 195

8. Implémenter ce réseau.
9. Entraîner le modèle puis l'évaluer.
10. Que remarquez-vous ?
11. Proposer des solutions pour gérer ce phénomène (dropout, régularisation L1 et régularisation L2).

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

1. Lire le fichier « Tweets.csv » et afficher son contenu :

```
import pandas as pd
import numpy as np
df = pd.read_csv('Tweets.csv')
df
```

```
tweet_id airline_sentiment airline_sentiment_confidence       negativereason
         negativereason_confidence airline  airline_sentiment_gold    name
         negativereason_gold       retweet_count    text     tweet_coord      tweet_created
         tweet_location   user_timezone
0        570306133677760513        neutral  1.0000   NaN      NaN      Virgin America   NaN
         cairdin  NaN      0        @VirginAmerica What @dhepburn said.        NaN      2015-02-
24 11:35:52 -0800NaN      Eastern Time (US & Canada)
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1: Gérer le surapprentissage

2. Conserver que la colonne « text » comme entrée et la colonne « airlines_sentiment » comme cible :

```
df = df[['text', 'airline_sentiment']]
```

3. Supprimer les stopwords et les mentions :

```
df.text = df.text.apply(remove_stopwords).apply(remove_mentions)
```

4. Séparer les données en 90% pour l'apprentissage et 10% restantes pour le test :

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.text, df.airline_sentiment,
test_size=0.1, random_state=42)
print('Données Apprentissage :', X_train.shape[0])
print('Données Test :', X_test.shape[0])
assert X_train.shape[0] == y_train.shape[0]
assert X_test.shape[0] == y_test.shape[0]
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

5. En utilisant la méthode **texts_to_matrix** du Tokenizer, convertir le texte d'un tweet en un vecteur avec des valeurs **NB_WORDS**. Avec mode = binaire, il contient un indicateur indiquant si le mot est apparu dans le tweet ou non :

```
X_train_oh = tk.texts_to_matrix(X_train, mode='binary')
X_test_oh = tk.texts_to_matrix(X_test, mode='binary')
```

6. Convertir les classes cibles en nombres en utilisant **LabelEncoder()** du Sklearn, puis appliquer le one-hot-encoding avec la méthode **to_categorical()** du Keras :

```
from sklearn.preprocessing import LabelEncoder
from keras.utils.np_utils import to_categorical

le = LabelEncoder()
y_train_le = le.fit_transform(y_train)
y_test_le = le.fit_transform(y_test)
y_train_oh = to_categorical(y_train_le)
y_test_oh = to_categorical(y_test_le)
print('"{}" est convertit en {}'.format(y_train[0], y_train_le[0]))
print('"{}" est convertit en {}'.format(y_train_le[0], y_train_oh[0]))
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

7. Séparer les données d'apprentissage en 90% de données d'apprentissage et 10% de données de validation. Cet ensemble de validation sera utilisé pour évaluer les performances du modèle :

```
X_train_rest, X_valid, y_train_rest, y_valid = train_test_split(X_train_oh, y_train_oh,
test_size=0.1, random_state=37)
assert X_valid.shape[0] == y_valid.shape[0]
assert X_train_rest.shape[0] == y_train_rest.shape[0]
```

8. Implémenter ce réseau :

```
from keras import models
from keras import layers
from keras import regularizers
base_model = models.Sequential()
base_model.add(layers.Dense(64, activation='relu', input_shape=(NB_WORDS,)))
base_model.add(layers.Dense(64, activation='relu'))
base_model.add(layers.Dense(3, activation='softmax'))
base_model._name = 'Modèle de base'
base_model.summary()
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

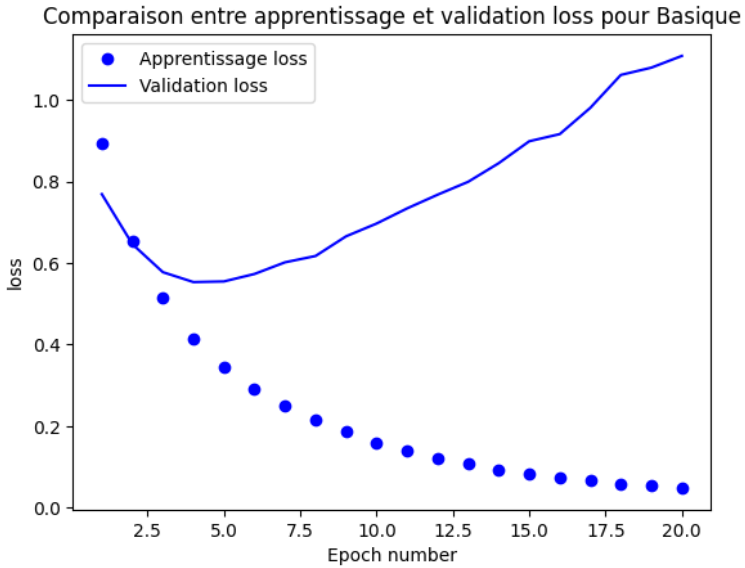
Exercice 1 : Gérer le surapprentissage

9. Entraîner le modèle puis l'évaluer :

```
base_history = deep_model(base_model, X_train_rest, y_train_rest, X_valid, y_valid)
base_min = optimal_epoch(base_history)
```

```
Perte de validation minimale atteinte à l'époque 4
```

```
eval_metric(base_model, base_history, 'loss')
```



PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

10. Que remarquez-vous ?

Le modèle rencontre un surapprentissage.

11. Proposer des solutions pour gérer ce phénomène (dropout, régularisation L_1 et régularisation L_2) :

```
reg_model = models.Sequential()
reg_model.add(layers.Dense(64,
kernel_regularizer=regularizers.l2(0.001),
activation='relu', input_shape=(NB_WORDS,)))
reg_model.add(layers.Dense(64,
kernel_regularizer=regularizers.l2(0.001),
activation='relu'))
reg_model.add(layers.Dense(3, activation='softmax'))
reg_model._name = 'L2_Regularisation'
reg_model.summary()
```

```
Model: "L2_Regularisation"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
dense_12 (Dense)            (None, 64)                640064

 dense_13 (Dense)            (None, 64)                4160

 dense_14 (Dense)            (None, 3)                 195

=================================================================
Total params: 644,419
Trainable params: 644,419
Non-trainable params: 0
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

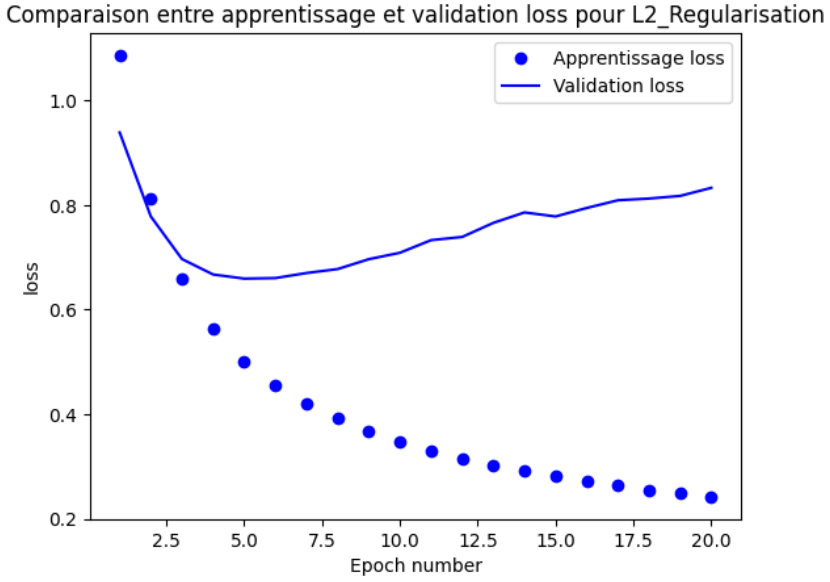
Correction

Exercice 1 : Gérer le surapprentissage

```
reg_history = deep_model(reg_model, X_train_rest, y_train_rest, X_valid, y_valid)
reg_min = optimal_epoch(reg_history)
```

```
Perte de validation minimale atteinte à l'époque 5
```

```
eval_metric(reg_model, reg_history, 'loss')
```



PARTIE 1

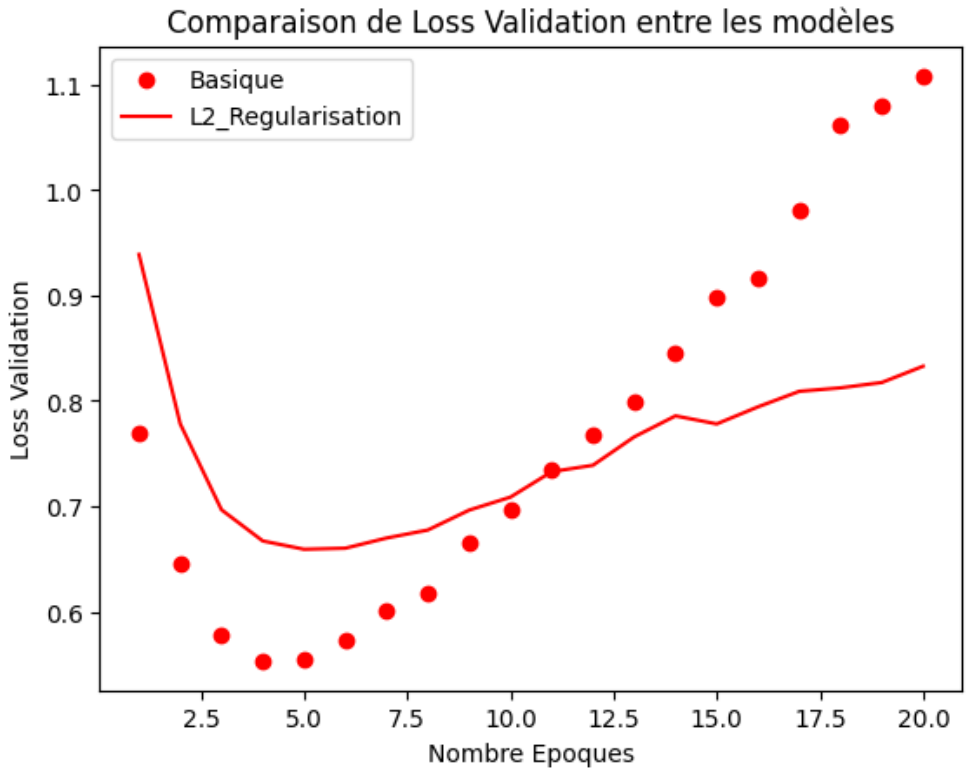
Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

```
compare_models_by_metric(base_model, reg_model, base_history, reg_history, 'val_loss')
```



PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

```
reg_model_L1 = models.Sequential()
reg_model_L1.add(layers.Dense(64,
kernel_regularizer=regularizers.l1(0.001),
activation='relu', input_shape=(NB_WORDS,)))
reg_model_L1.add(layers.Dense(64,
kernel_regularizer=regularizers.l1(0.001),
activation='relu'))
reg_model_L1.add(layers.Dense(3,
activation='softmax'))
reg_model_L1._name = 'L1_Regularisation'
reg_model_L1.summary()
```

```
Model: "L1_Regularisation"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
dense_15 (Dense)             (None, 64)                640064

 dense_16 (Dense)            (None, 64)                4160

 dense_17 (Dense)            (None, 3)                 195

=================================================================
Total params: 644,419
Trainable params: 644,419
Non-trainable params: 0
_________________________________________________________________
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

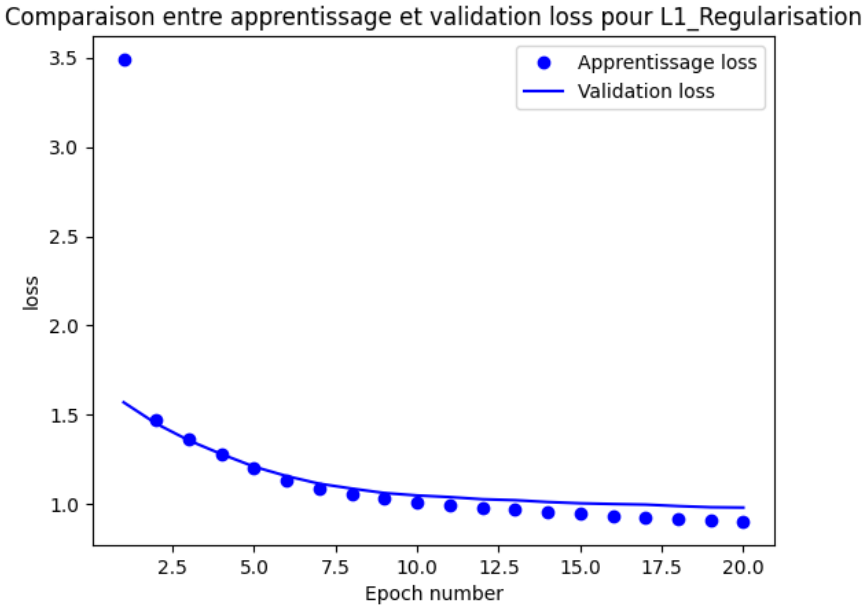
Correction

Exercice 1 : Gérer le surapprentissage

```
reg_history_L1 = deep_model(reg_model_L1, X_train_rest, y_train_rest, X_valid, y_valid)
reg_min_L1 = optimal_epoch(reg_history_L1)
```

```
Perte de validation minimale atteinte à l'époque 20
```

```
eval_metric(reg_model_L1, reg_history_L1, 'loss')
```



PARTIE 1

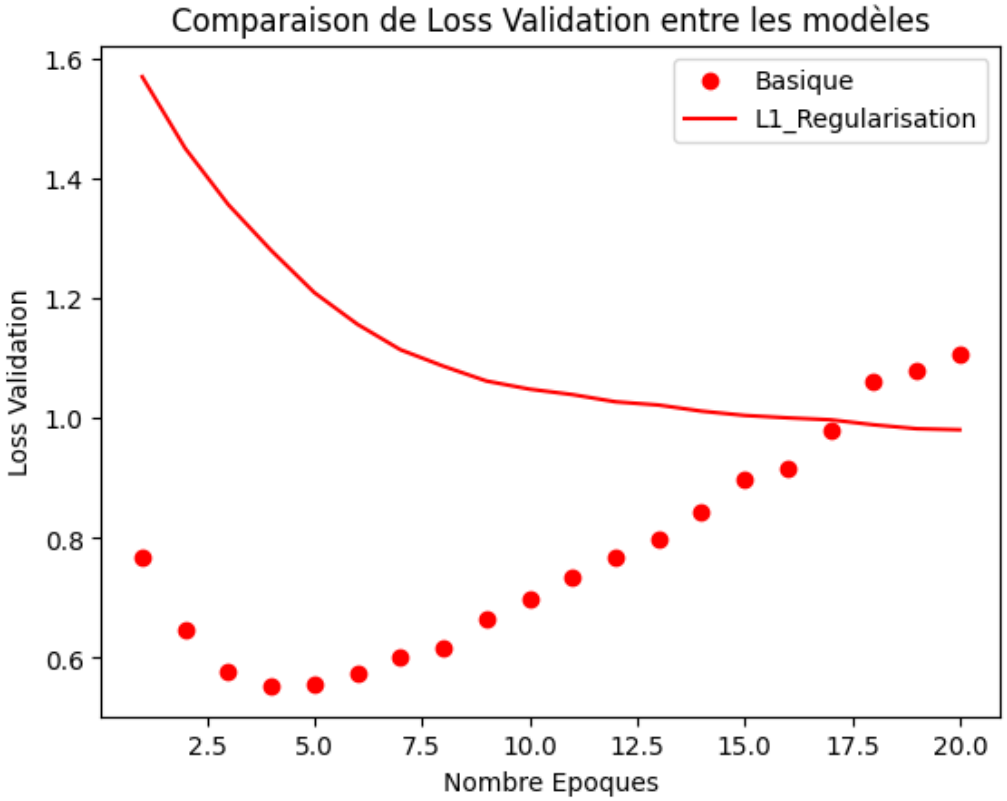
Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

```
compare_models_by_metric(base_model, reg_model_L1, base_history, reg_history_L1, 'val_loss')
```



PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

Exercice 1 : Gérer le surapprentissage

```
drop_model = models.Sequential()
drop_model.add(layers.Dense(64, activation='relu',
input_shape=(NB_WORDS,)))
drop_model.add(layers.Dropout(0.5))
drop_model.add(layers.Dense(64, activation='relu'))
drop_model.add(layers.Dropout(0.5))
drop_model.add(layers.Dense(3, activation='softmax'))
drop_model._name = 'Dropout'
drop_model.summary()
```

```
Model: "Dropout"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_18 (Dense)            (None, 64)                640064

 dropout (Dropout)           (None, 64)                0

 dense_19 (Dense)            (None, 64)                4160

 dropout_1 (Dropout)         (None, 64)                0

 dense_20 (Dense)            (None, 3)                 195

=================================================================
Total params: 644,419
Trainable params: 644,419
Non-trainable params: 0
_________________________________________________________________
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Activité 2

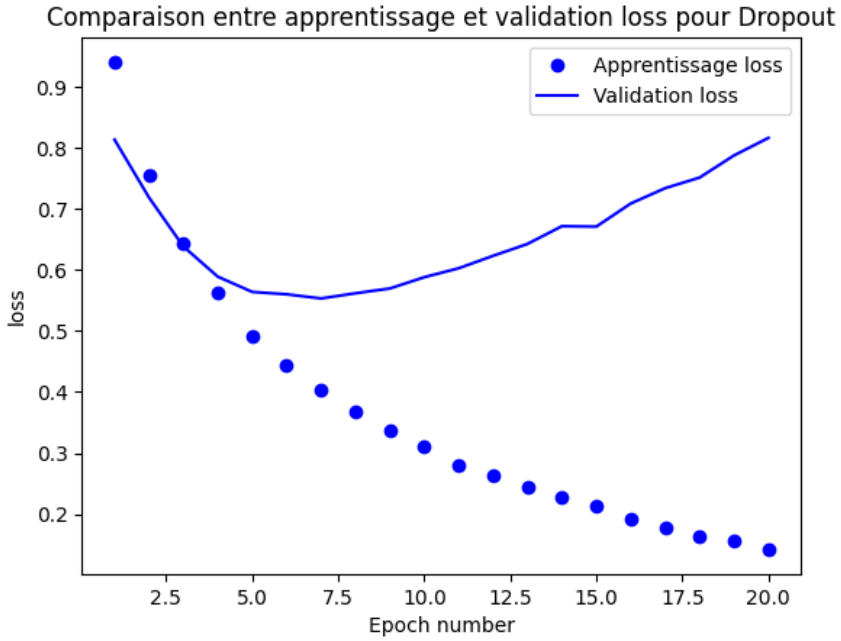
Correction

Exercice 1 : Gérer le surapprentissage

```
drop_history = deep_model(drop_model, X_train_rest, y_train_rest, X_valid, y_valid)
drop_min = optimal_epoch(drop_history)
```

```
Perte de validation minimale atteinte à l'époque 7
```

```
eval_metric(drop_model, drop_history, 'loss')
```



PARTIE 1

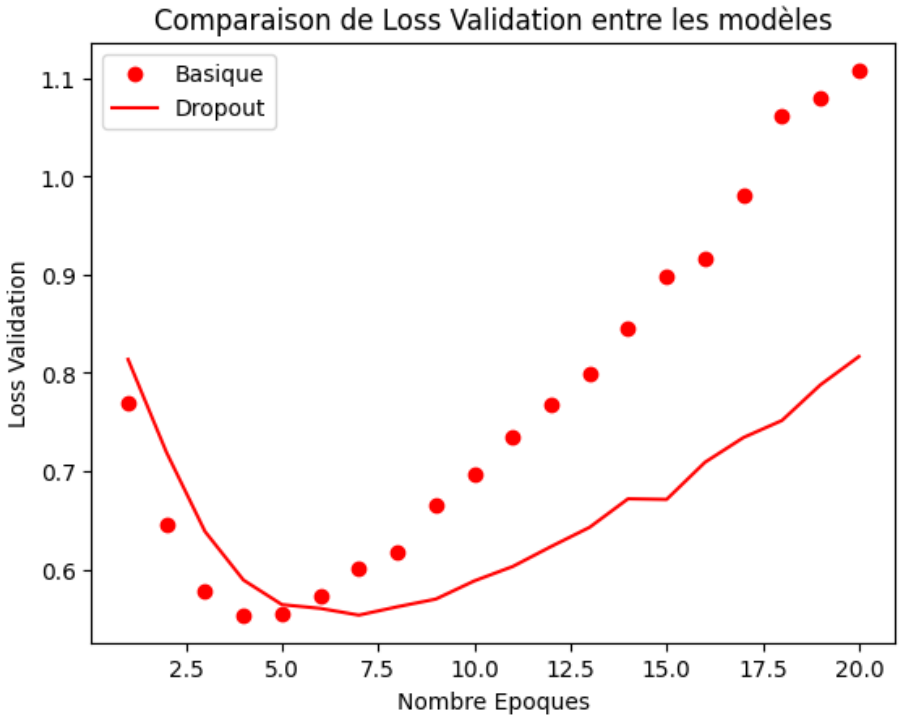
Copyright - Tout droit réservé - OFPPT

Activité 2

Correction

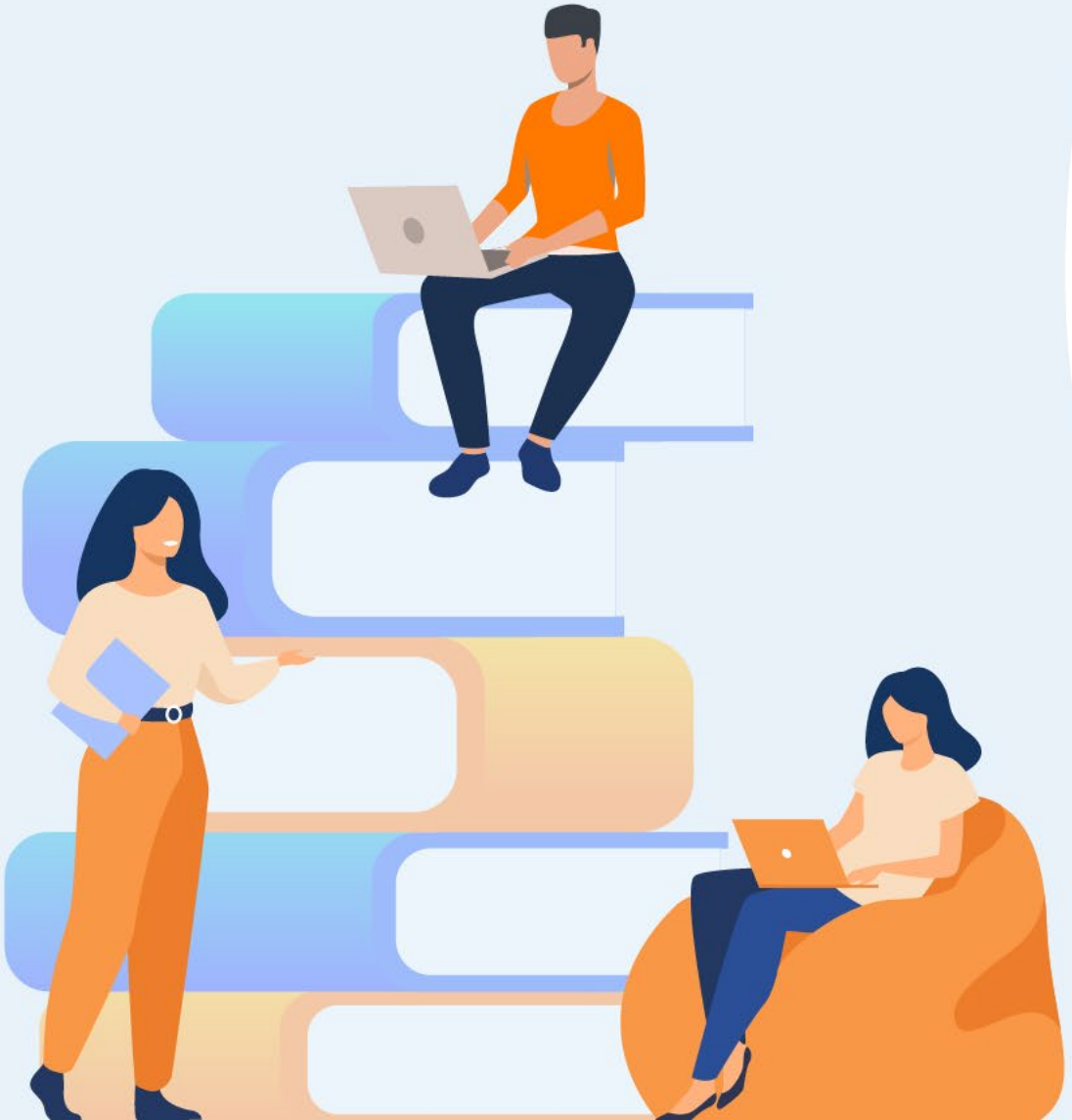
Exercice 1 : Gérer le surapprentissage

```
compare_models_by_metric(base_model, drop_model, base_history, drop_history, 'val_loss')
```



PARTIE 1

Copyright - Tout droit réservé - OFPPT



PARTIE 2

MANIPULER LES RÉSEAUX DE NEURONES AVANCÉS

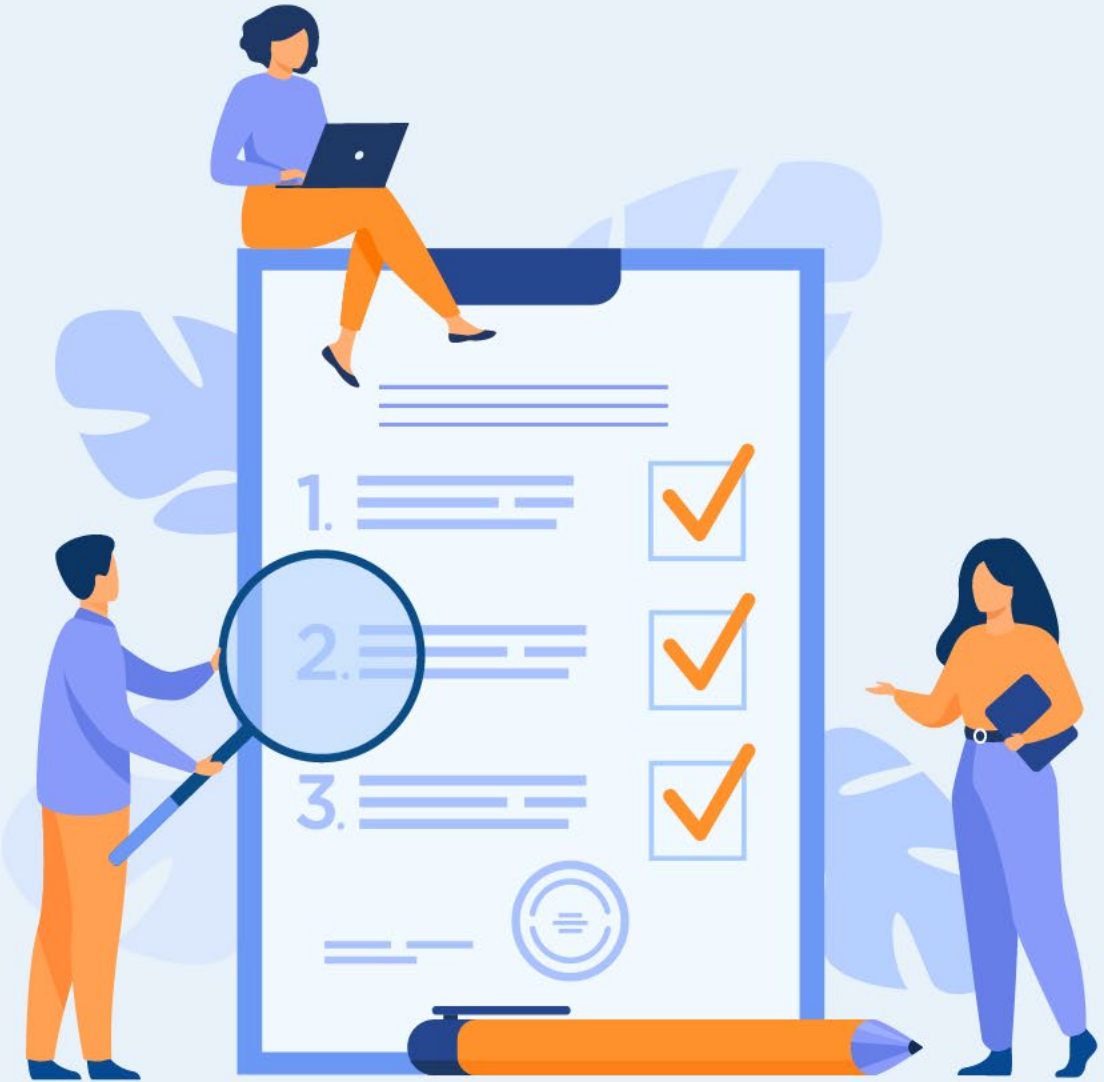
Dans ce module, vous allez :

- Construire des réseaux à convolution
- Optimiser les réseaux à convolution

15 heures

Activité 1

CONSTRUIRE ET OPTIMISER DES RÉSEAUX À CONVOLUTION



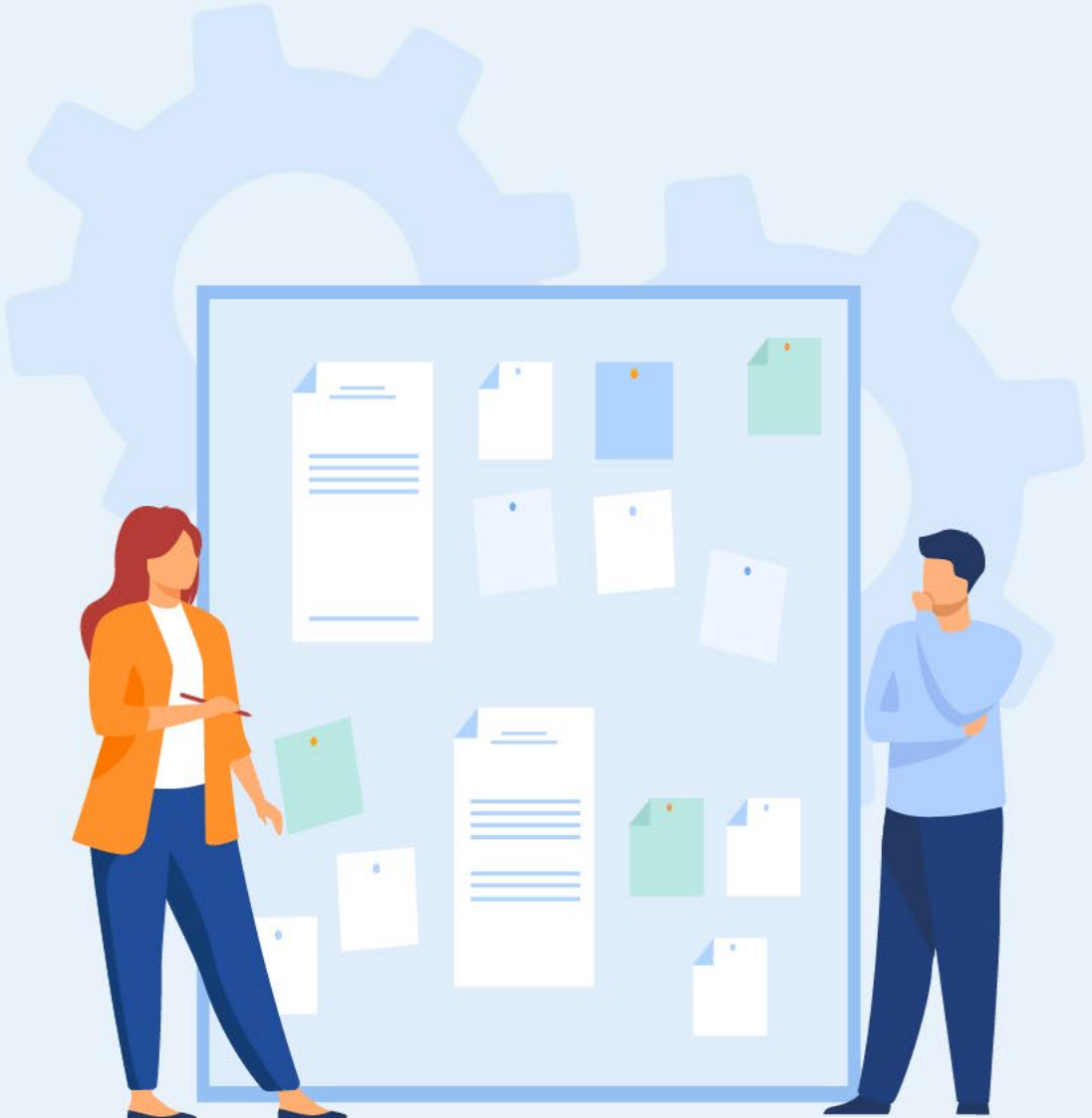
Compétences visées :

- Implémenter les différents scénarios d'apprentissage par transfert
- Évaluer les différents scénarios d'apprentissage par transfert

Recommandations clés :

- Lire attentivement l'énoncé de l'exercice
- Utiliser Keras pour implémenter les réseaux de neurones à convolution

15 heures



CONSIGNES

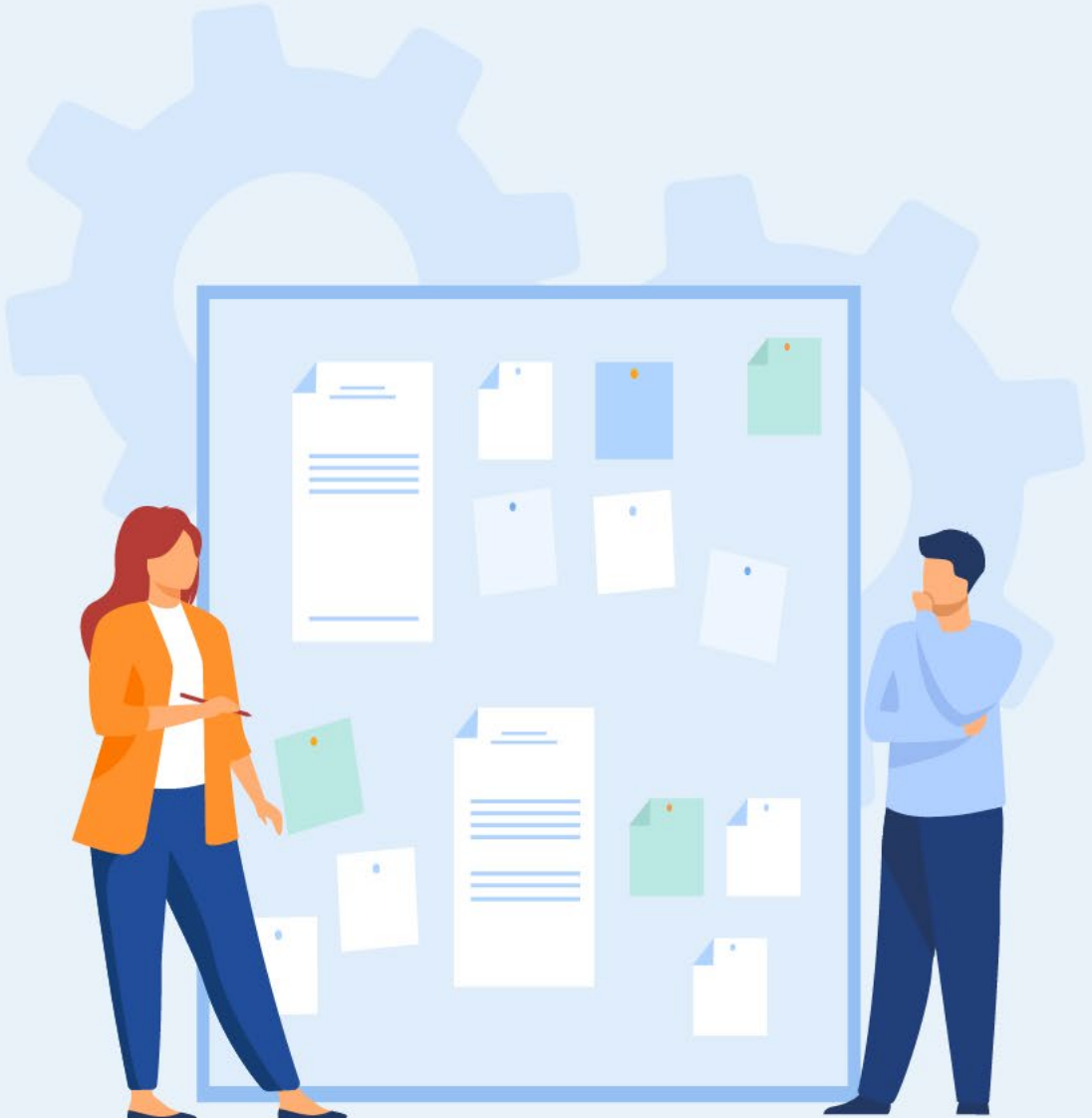
1- Pour le formateur

- Demander d’implémenter les différents scénarios d’apprentissage par transfert

2- Pour l’apprenant

- Implémenter les différents scénarios d’apprentissage par transfert Tracer la droite de la régression

CONSIGNES



3 - Conditions de réalisation

- Utilisation des supports pédagogiques fournis par le formateur
- Suivi des instructions verbales ou écrites du formateur
- Travail individuel ou en groupe
- Utilisation d’un ordinateur
- Utilisation du Google Colab
- Acquisition d’une connexion internet

4 - Critères de réussite

- Le stagiaire est-il capable de :
  - Implémenter les scénarios d’apprentissage par transfert ?

Activité 1

Exercices

Exercice 1 : CNN comme extracteur de caractéristiques

Cette stratégie consiste à se servir des caractéristiques du réseau pré-entraîné pour représenter les images du nouveau problème. Pour cela, on retire la dernière couche fully-connected et on fixe tous les autres paramètres. Ce réseau va ainsi calculer la représentation de chaque image en entrée à partir des caractéristiques déjà apprises lors du pré-entraînement.

La stratégie doit être utilisée lorsque la nouvelle collection d'images est petite et similaire aux images de pré-entraînement. En effet, entraîner le réseau sur peu d'images est dangereux puisque le risque de surapprentissage est important. De plus, si les nouvelles images ressemblent aux anciennes, elles peuvent alors être représentées par les mêmes caractéristiques.

1. Implémenter la fonction **create_model_descriptor()** qui crée un model CNN de type VGG16 entraîné sur la base de données Imagenet. Dans cette fonction, inclure que la base du VGG16 et supprimer la dernière couche de sortie. Le but de cette fonction est d'extraire les caractéristiques du VGG16(). Cette fonction retourne vgg_descriptor qui contient les caractéristiques extraites.

La fonction **preprocess_image()** joue un rôle important pour l'adaptation de l'image à l'entrée du modèle VGG16. L'image est convertie en numpy array, puis redimensionnée. Ensuite, cette image est convertie du RGB en BGR puis centrée à zéro. Exécuter ce bout de code :

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1 : CNN comme extracteur de caractéristiques

```
from tensorflow.keras.utils import load_img
from tensorflow.keras.utils import img_to_array
from keras.applications.vgg16 import preprocess_input

def preprocess_image(image_path):
    img = load_img(image_path, target_size=(224, 224))
    img = img_to_array(img)
    img = img.reshape((1, img.shape[0], img.shape[1], img.shape[2])) # to shape (1,n,m,3)
    img = preprocess_input(img)# to convert it from RGB to BGR then each color channel is
zero-centered with respect to the ImageNet dataset
    return img
```

2. Implémenter la fonction **get_image_presentation()** qui extrait les caractéristiques d'une image donnée en entrée et prétraitée avec la fonction **preprocess_image()**. Cette fonction prend comme paramètres vgg_descriptor et l'image.

3. Implémenter la fonction **findEuclideanDistance()** qui permet de calculer la distance euclidienne entre deux descripteurs VGG16 de deux images données en entrée. Cette fonction prend comme paramètres source_representation et test_representation qui représentent les deux vecteurs de caractéristiques.

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1 : CNN comme extracteur de caractéristiques

La fonction **verifyObject()** vérifie si les deux images sont similaires ou non en fixant un taux d'acceptation epsilon=90.

N.B: epsilon n'est pas forcément la valeur optimale d'acceptation ou de rejet du test. Exécuter ce bout de code :

```
def verifyObject(vgg_descriptor, img1, img2, epsilon=90):
    img1_representation = vgg_descriptor.predict(preprocess_image(img1))[0,:]
    img2_representation = vgg_descriptor.predict(preprocess_image(img2))[0,:]
    euclidean_distance = findEuclideanDistance(img1_representation, img2_representation)
    print("Distance euclidienne: ",euclidean_distance)
    if(euclidean_distance < epsilon):
        print("Vérifié! Ce sont les mêmes objets")
    else:
        print("Non vérifié! Les objets sont différents")
```

4. Appeler toutes ces fonctions dans la fonction main() pour tester l'extraction des caractéristiques sur les images dog1.jpg, dog2.jpg, mug1.jpg et mug2.jpg.

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 2 : CNN pré-entraîné

On charge tout le modèle VGG16 entraîné sur la base de données Imagenet. Dans ce cas, on utilise également les couches de sortie car le but de cet exercice est de prédire la classe d'une image similaire à celles de la base de données Imagenet.

1. Charger le modèle VGG16 avec les poids pré-entraînés Imagenet.
2. Charger une image de votre Drive (mug1.jpg par exemple), puis appliquer le prétraitement comme dans l'exercice 1.
3. Prédire la classe de l'image « mug1.jpg" après le prétraitement.
4. Transformer les probabilités en des labels, puis afficher les classes les plus probables.

Copyright - Tout droit réservé - OFPPT

Exercice 3 : Réajustement du CNN (finetuning)

On remplace à nouveau la dernière couche fully-connected par le nouveau classifieur initialisé aléatoirement, et on fixe les paramètres de certaines couches du réseau pré-entraîné. Ainsi, en plus du classifieur, on entraîne sur les nouvelles images les couches qui correspondent en général aux plus profondes du réseau. On utilise cette stratégie lorsque la nouvelle collection d'images est petite mais très différente des images du pré-entraînement.

Dans cet exercice, on va utiliser la base de données « Flowers » qui va être partagée via le drive.

La base de données Flowers est téléchargeable à partir du lien suivant: https://www.kaggle.com/alxmamaev/flowers-recognition. Cette dernière représente les images de cinq classes de fleurs: daisy, dandelion, rose, sunflower, et tulips. Après avoir téléchargé la base de données, on a sélectionné que trois classes parmi les cinq afin d'entrainer un nombre limité d'images. Ensuite, on a séparé les données deux dossiers - «train» et «test». Ceux-ci contiennent respectivement l'ensemble d'entraînement et de test. À l'intérieur de chaque dossier, il y a des dossiers séparés pour chaque classe.

Copyright - Tout droit réservé - OFPPT

Exercice 3 : Réajustement du CNN (finetuning)

On charge tout d'abord la base de données en utilisant **ImageDataGenerator** de Keras. Exécuter ce bout de code :

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_datagen = ImageDataGenerator(
      rescale=1./255,
      rotation_range=20,
      width_shift_range=0.2,
      height_shift_range=0.2,
      horizontal_flip=True,
      fill_mode='nearest')
validation_datagen = ImageDataGenerator(rescale=1./255)
batchsize = 16
train_generator = train_datagen.flow_from_directory(
        '/content/drive/My Drive/flowers/train',
        target_size=(224, 224),
        batch_size=batchsize,
        class_mode='categorical')
validation_generator = validation_datagen.flow_from_directory(
        '/content/drive/My Drive/flowers/test',
        target_size=(224, 224),
        batch_size=batchsize,
        class_mode='categorical')
```


Copyright - Tout droit réservé - OFPPT

Exercice 3 : Réajustement du CNN (finetuning)

1. Quel est le nombre d'images d'entraînement et de test ?
2. Charger le modèle VGG16 en entraînant que les quatre dernières couches.
3. En plus du modèle VGG16 chargé, ajouter une couche flatten, une couche fully connected de 1024 neurones, une couche dropout et la couche de sortie qui contient 3 neurones pour représenter les trois classes.
4. Entraîner le modèle en exécutant ce bout de code. Que remarquez-vous sur l'évolution de l'entraînement ?

```
from tensorflow.keras import optimizers
model.compile(loss='categorical_crossentropy',
              optimizer=optimizers.SGD(learning_rate=1e-4),
              metrics=['acc'])
history = model.fit_generator(
      train_generator,
      steps_per_epoch=train_generator.samples/train_generator.batch_size ,
      epochs=30,
      validation_data=validation_generator,
      validation_steps=validation_generator.samples/validation_generator.batch_size,
      verbose=1)
```


Copyright - Tout droit réservé - OFPPT

Exercice 3 : Réajustement du CNN (finetuning)

5. Afficher l'historique de l'entraînement en exécutant ce bout de code :

```
import matplotlib.pyplot as plt
acc = history.history['acc']
val_acc = history.history['val_acc']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(len(acc))
plt.plot(epochs, acc, 'b', label='Entrainement')
plt.plot(epochs, val_acc, 'r', label='Validation')
plt.title('Accuracy')
plt.legend()
plt.figure()
plt.plot(epochs, loss, 'b', label='Entrainement')
plt.plot(epochs, val_loss, 'r', label='Validation')
plt.title('Loss')
plt.legend()
plt.show()
```


Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1 : CNN comme extracteur de caractéristiques

1. Implémenter la fonction create_model_descriptor() qui crée un model CNN de type VGG16 entrainé sur la base de données Imagenet :

```
from keras.applications.vgg16 import VGG16
from keras.models import Model
def create_model_descriptor():
    vgg_descriptor = VGG16()
    vgg_descriptor = Model(inputs=vgg_descriptor.inputs, outputs=vgg_descriptor.layers[-
2].output)
    vgg_descriptor.summary()
    return vgg_descriptor
```

2. Implémenter la fonction **get_image_presentation()** qui extrait les caractéristiques d'une image donnée en entrée et prétraitée avec la fonction **preprocess_image()** :

```
def get_img_representation(vgg_descriptor, img):
    return vgg_descriptor.predict(preprocess_image(img))[0,:]
```

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1 : CNN comme extracteur de caractéristiques

3. Implémenter la fonction **findEuclideanDistance()** qui permet de calculer la distance euclidienne entre deux descripteurs VGG16 de deux images données en entrée :

```
import numpy as np
def findEuclideanDistance(source_representation, test_representation):
    euclidean_distance = source_representation - test_representation
    euclidean_distance = np.sum(np.multiply(euclidean_distance, euclidean_distance))
    euclidean_distance = np.sqrt(euclidean_distance)
    return euclidean_distance
```

4. Appeler toutes ces fonctions dans la fonction main() pour tester l'extraction des caractéristiques sur les images dog1.jpg, dog2.jpg, mug1.jpg et mug2.jpg :

```
from google.colab import drive
drive.mount('/content/drive')
```

```
if __name__ == "__main__":
    vgg_descriptor = create_model_descriptor()
    verifyObject(vgg_descriptor, "/content/drive/MyDrive/TP CNN/dog2.jpg",
"/content/drive/MyDrive/TP CNN/dog1.jpg")
```

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 2 : CNN pré-entraîné

1. Charger le modèle VGG16 avec les poids pré-entraînés Imagenet :

```
model = VGG16(weights='imagenet')
model.summary()
```

2. Charger une image de votre Drive (mug1.jpg par exemple), puis appliquer le prétraitement comme dans l'exercice 1 :

```
image = load_img('/content/drive/MyDrive/TP CNN/mug1.jpg', target_size=(224, 224))
image = img_to_array(image)
image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
image = preprocess_input(image)
```

3. Prédire la classe de l'image « mug1.jpg" après le prétraitement :

```
from keras.applications.vgg16 import decode_predictions
yhat = model.predict(image)
print('La taille du vecteur est',yhat.shape)
```

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 2 : CNN pré-entraîné

4. Transformer les probabilités en des labels, puis afficher les classes les plus probables :

```
label = decode_predictions(yhat)
print(label)
label = label[0][0]
print('%s (%.2f%%)' % (label[1], label[2]*100))
```

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 3 : Réajustement du CNN (finetuning)

1. Quel est le nombre d'images d'entraînement et de test ?

```
Found 540 images belonging to 3 classes.
Found 108 images belonging to 3 classes.
```

2. Charger le modèle VGG16 en entraînant que les quatre dernières couches :

```
from keras.applications.vgg16 import VGG16
vgg_conv = VGG16(weights="imagenet", include_top=False, input_shape=(224, 224, 3))
for layer in vgg_conv.layers[:-4]:
    layer.trainable = False
for layer in vgg_conv.layers:
    print(layer, layer.trainable)
```

3. En plus du modèle VGG16 chargé, ajouter une couche flatten, une couche fully connected de 1024 neurones, une couche dropout et la couche de sortie qui contient 3 neurones pour représenter les trois classes :

PARTIE 2

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 3 : Réajustement du CNN (finetuning)

```
from keras.layers import Dense
from keras import models
from keras import layers
model = models.Sequential()
model.add(vgg_conv)
model.add(layers.Flatten())
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(3, activation='softmax'))
model.summary()
```

4. Entraîner le modèle en exécutant ce bout de code. Que remarquez-vous sur l'évolution de l'entraînement ?

```
Epoch 1/30
33/33 [==============================] - 344s 10s/step - loss: 1.2373 - acc: 0.3741 - val_loss: 1.1001 -
val_acc: 0.3333
Epoch 2/30
33/33 [==============================] - 9s 272ms/step - loss: 1.1464 - acc: 0.3574 - val_loss: 1.0770 -
val_acc: 0.3796
Epoch 3/30
33/33 [==============================] - 10s 281ms/step - loss: 1.0923 - acc: 0.4000 - val_loss: 1.0406 -
val_acc: 0.4259
```

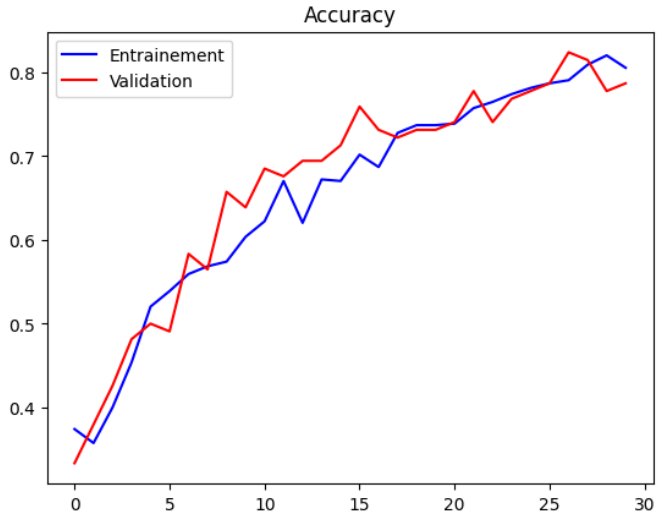
PARTIE 2

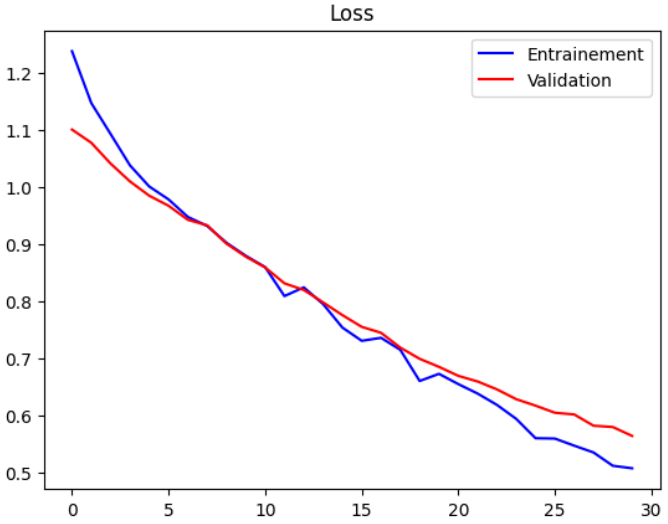
Copyright - Tout droit réservé - OFPPT

Exercice 3 : Réajustement du CNN (finetuning)

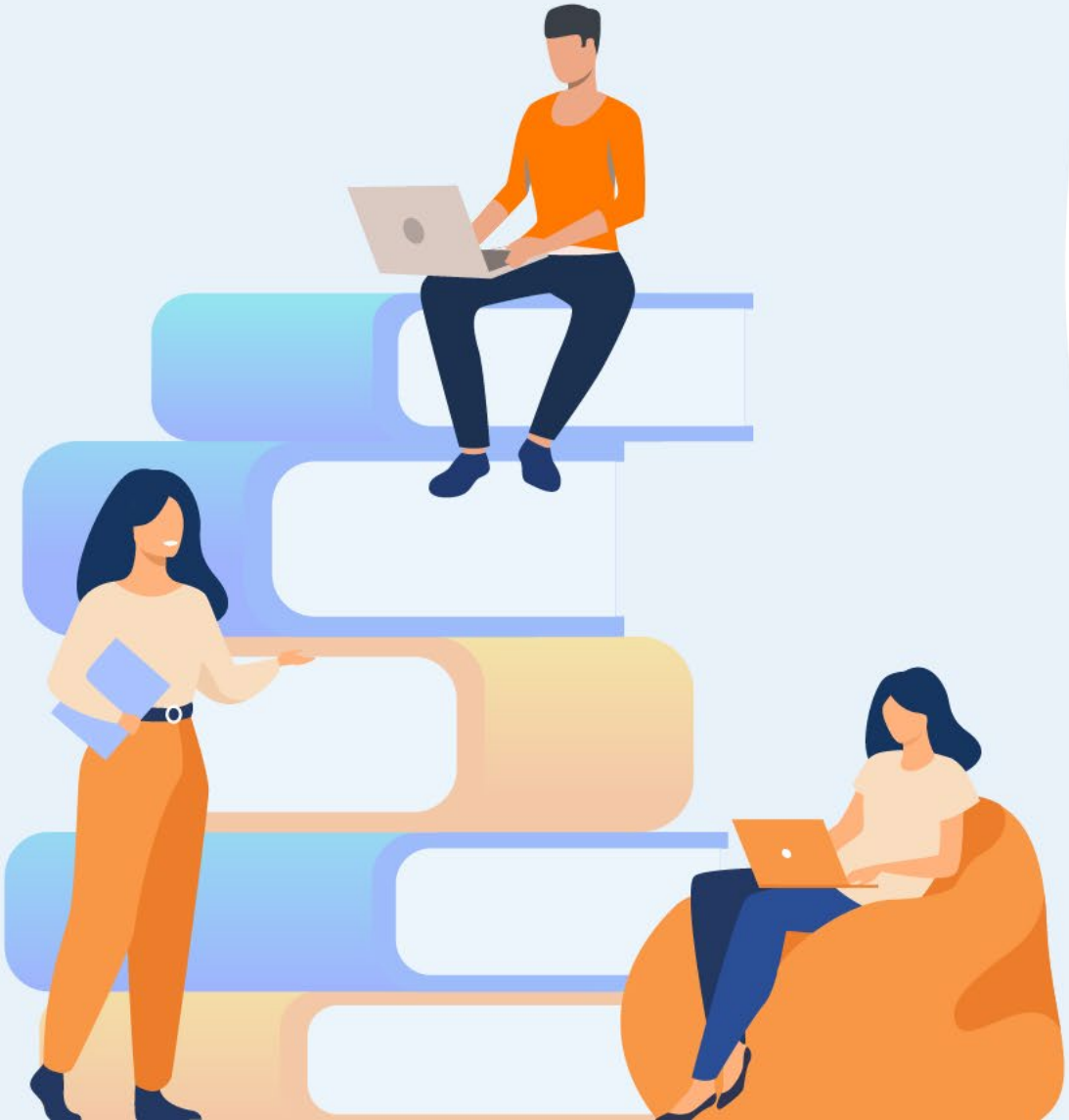
On remarque que durant l'entraînement l'accuracy de l'entraînement ainsi que celui de la validation augmentent après chaque époque. La valeur de la perte (loss) pour les deux cas entraînement et validation diminue. On remarque également que pendant 30 époques la qualité du modèle s'améliore. On peut lancer le modèle à 100 époques pour obtenir un modèle plus performant.

5. Afficher l'historique de l'entraînement en exécutant ce bout de code :





Copyright - Tout droit réservé - OFPPT



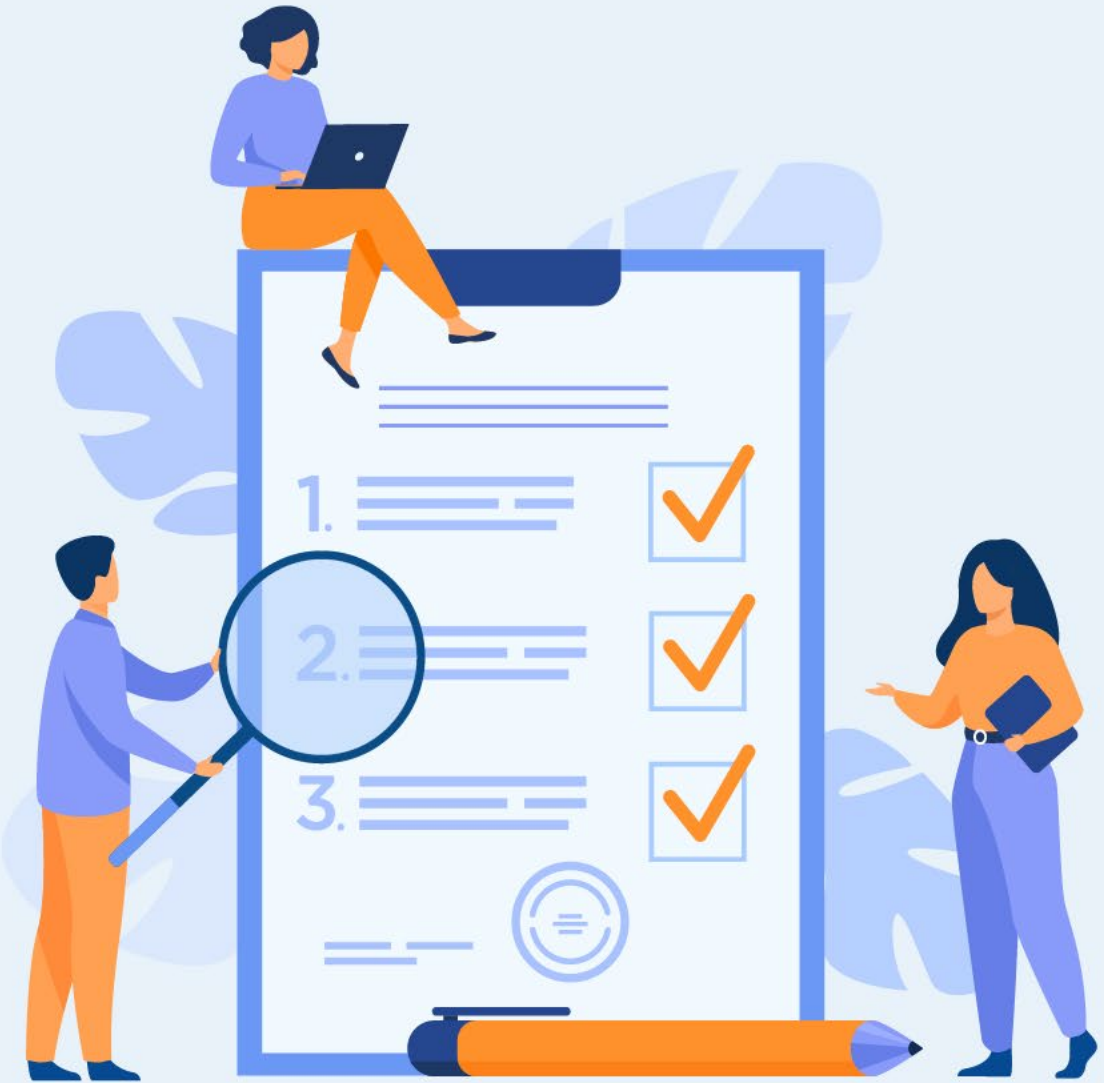
PARTIE 3

EXPLORER LES RÉSEAUX RÉCURRENTS

Dans ce module, vous allez :

- Comprendre les réseaux récurrents
- Appliquer les réseaux de mémoire à long terme (LSTM) et les unités récurrentes à portes (GRU)

15 heures



Activité 1

COMPRENDRE LES RÉSEAUX RÉCURRENTS & APPLIQUER LSTM ET GRU

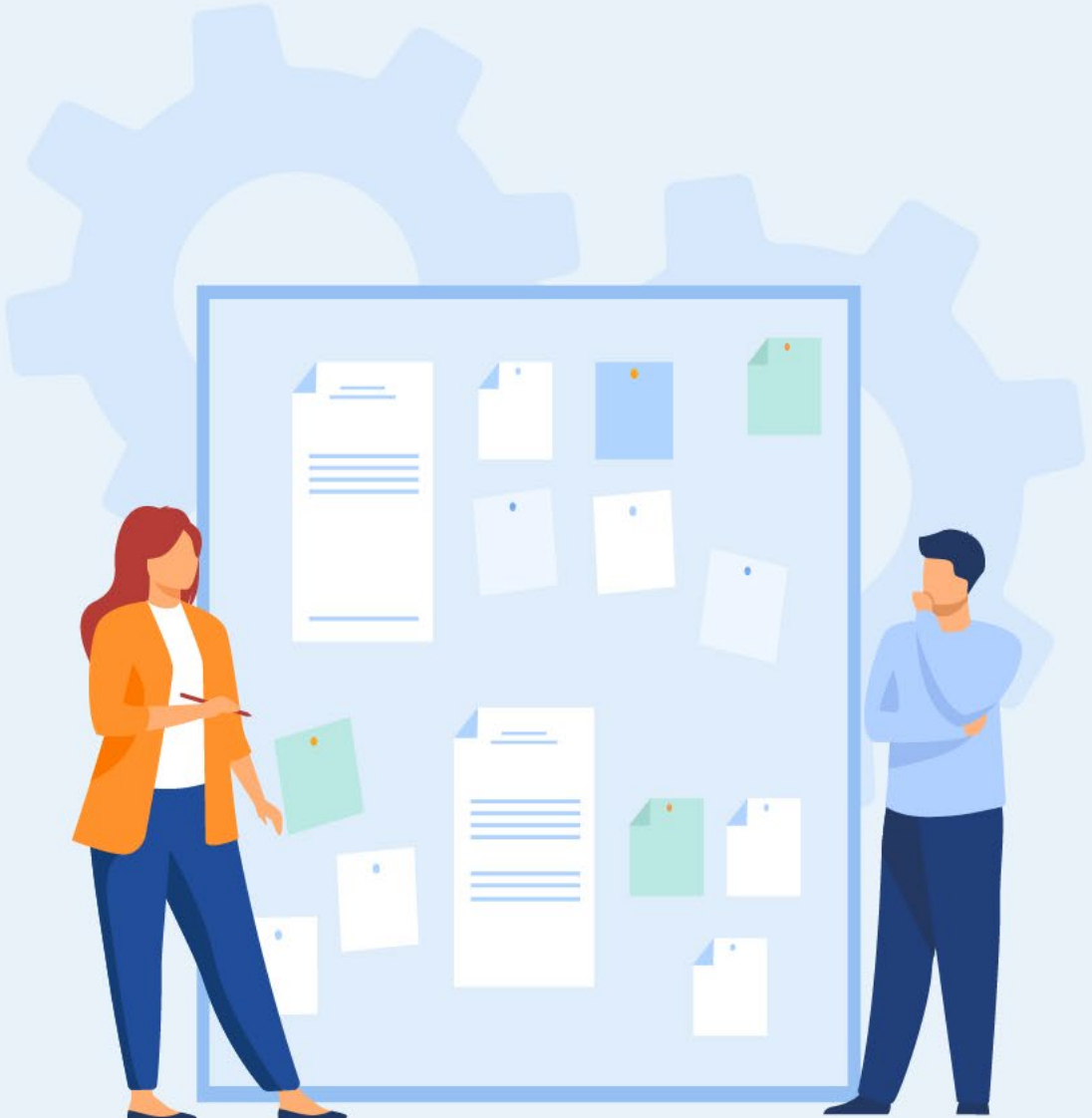
Compétences visées :

- Appliquer les réseaux récurrents pour les prévisions

Recommandations clés :

- Suivre convenablement les consignes du formateur

15 heures



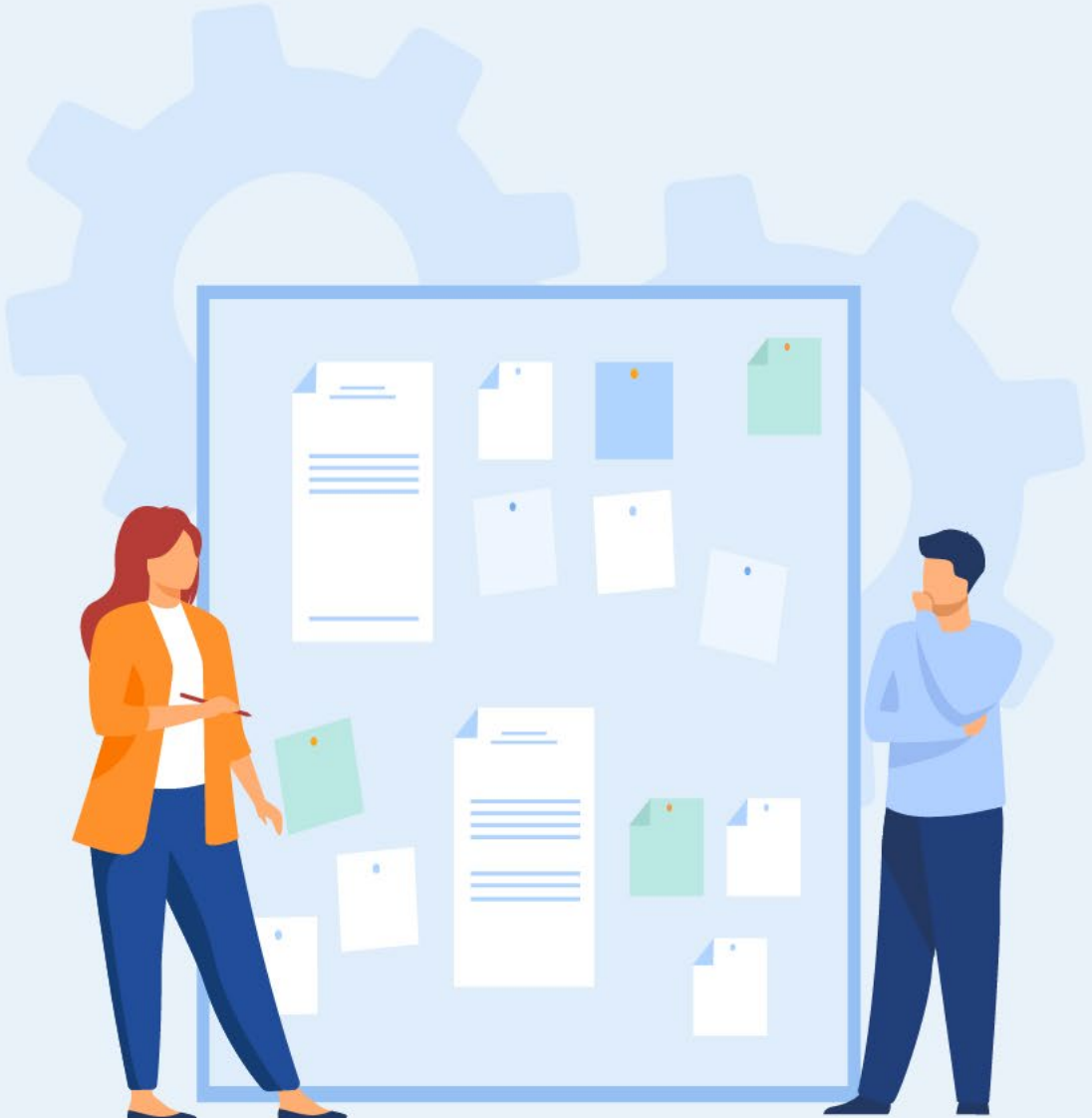
CONSIGNES

1- Pour le formateur

- Demander d’implémenter un réseau récurrent
- Demander d’appliquer un réseau récurrent pour les prévisions

2- Pour l’apprenant

- Implémenter un réseau récurrent
- Appliquer un réseau récurrent pour les prévisions



CONSIGNES

3 - Conditions de réalisation

- Utilisation des supports pédagogiques fournis par le formateur
- Suivi des instructions verbales ou écrites du formateur
- Travail individuel ou en groupe
- Utilisation d’un ordinateur
- Utilisation du Google Colab
- Acquisition d’une connexion internet

4 - Critères de réussite

- Le stagiaire est-il capable de :
  - Appliquer les réseaux récurrents ?

Activité 1

Exercices

Exercice 1 : Prédire les passagers des compagnies aériennes

1. Charger le fichier « AirPassengers.csv » sur Google Colab. Cet ensemble de données contient des informations sur les totaux mensuels de passagers d'une compagnie aérienne américaine de 1949 à 1960.
2. Lire ce fichier en utilisant la bibliothèque Pandas, puis afficher l'en tête du DataFrame.
3. Sélectionner la colonne « Month » et la convertir en un objet « datetime », ensuite modifier l'index pour qu'il devienne la colonne « Month ».
4. Afficher une courbe en utilisant matplotlib qui représente le nombre de passagers par mois.
5. Normaliser la colonne des passagers.
6. Exécuter ce bout de code pour séparer les données en 75% d'apprentissage et 25% de test :

```
train = int(len(data)*0.75)
test = len(data)-train
train_data,test_data=data[0:train,:],data[train:len(data),:1]
```

PARTIE 3

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1 : Prédire les passagers des compagnies aériennes

On définit une fonction **create_dataset()** qui permet d'organiser les données :

```
import numpy as np
def create_dataset(dataset, time_step=1):
        dataX, dataY = [], []
        for i in range(len(dataset)-time_step-1):
                a = dataset[i:(i+time_step), 0]
                dataX.append(a)
                dataY.append(dataset[i + time_step, 0])
        return np.array(dataX), np.array(dataY)
```

X=t	t+1	t+2	t+3
y=t+4			

Dans ce cas, on utilise 4 retours en arrière, donc une entrée X aura des valeurs de t-4 à t-1, et le y aura la valeur à t. Plus le timestep est grand, mieux il prédira les données d'entraînement (le timestep est également un hyperparamètre qu'on doit bien optimiser).

```
time_step = 4
X_train, y_train = create_dataset(train_data, time_step)
X_test, ytest = create_dataset(test_data, time_step)
```

PARTIE 3

Copyright - Tout droit réservé - OFPPT

Activité 1

Exercices

Exercice 1

7. Quel est le shape de X_train et X_test ?
8. Modifier le shape de X_train et X_test pour qu'il soit compatible avec l'entrée d'un réseau de neurones récurrent (nombre_d'échantillons,timestep,dimension_de_l'entrée)
9. Créer un modèle qui contient les couches suivantes :
   - ✓ Une couche LSTM avec 50 unités
   - ✓ Une couche LSTM avec 50 unités
   - ✓ Une couche Dense
10. Compiler et entraîner le modèle pendant 100 époques.
11. Prédire les résultats sur les données de test.
12. Calculer l'erreur quadratique moyenne.
13. Annuler la mise à l'échelle de ytest et le vecteur de prédiction.
14. Afficher la courbe des nombres de passagers prédits ainsi que les vraies valeurs des passagers.

PARTIE 3

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

1. Charger le fichier « AirPassengers.csv » sur Google Colab. Cet ensemble de données contient des informations sur les totaux mensuels de passagers d'une compagnie aérienne américaine de 1949 à 1960 :

```
from google.colab import files
files.upload()
```

2. Lire ce fichier en utilisant la bibliothèque Pandas, puis afficher l'en tête du DataFrame :

```
import pandas as pd
data = pd.read_csv('AirPassengers.csv')
data.head()
```

3. Sélectionner la colonne « Month » et la convertir en un objet « datetime », ensuite modifier l'index pour qu'il devienne la colonne « Month »:

```
data['Month'] = pd.to_datetime(data['Month'])
data.set_index('Month', inplace=True)
```

PARTIE 3

Copyright - Tout droit réservé - OFPPT

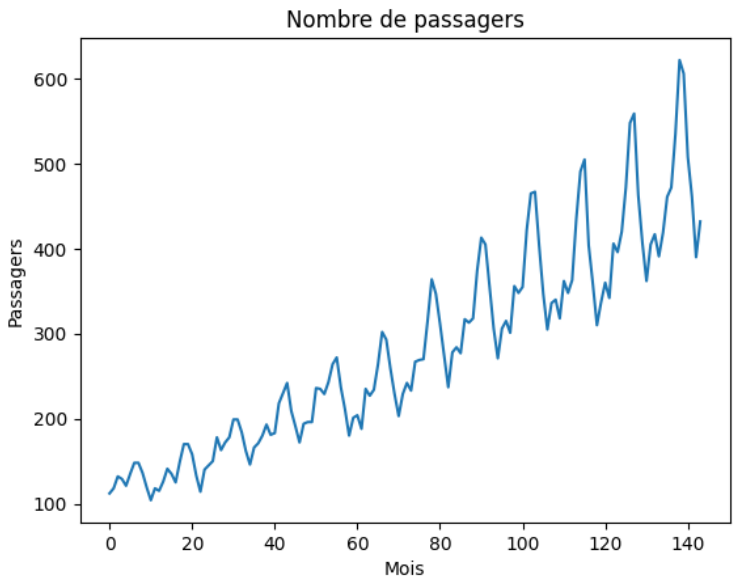
Activité 1

Correction

Exercice 1

4. Afficher une courbe en utilisant matplotlib qui représente le nombre de passagers par mois :

```
import matplotlib.pyplot as plt
plt.xlabel("Mois")
plt.ylabel("Passagers")
plt.title("Nombre de passagers")
plt.plot(data['#Passengers'],)
```



5. Normaliser la colonne des passagers :

```
data['#Passengers']
from sklearn.preprocessing import MinMaxScaler
scaler=MinMaxScaler(feature_range=(0,1))
data=scaler.fit_transform(data)
```

PARTIE 3

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

7. Quel est le shape de X_train et X_test ?

```
X_train.shape, y_train.shape
```

```
((103, 4), (103,))
```

8. Modifier le shape de X_train et X_test pour qu'il soit compatible avec l'entrée d'un réseau de neurones récurrent (nombre_d'échantillons,timestep,dimension_de_l'entrée) :

```
X_train =X_train.reshape(X_train.shape[0], 4, 1)
X_test = X_test.reshape(X_test.shape[0], 4, 1)
print(X_train.shape)
print(X_test.shape)
```

```
(103, 4, 1)
(31, 4, 1)
```

9. Créer un modèle qui contient les couches suivantes :

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,LSTM
model=Sequential()
model.add(LSTM(50,return_sequences=True,input_shape=(4,1)))
model.add(LSTM(50))
model.add(Dense(1))
```

PARTIE 3

Copyright - Tout droit réservé - OFPPT

Activité 1

Correction

Exercice 1

10. Compiler et entraîner le modèle pendant 100 époques :

```
model.compile(loss='mean_squared_error',optimizer='adam')
model.fit(X_train,y_train,validation_data=(X_test,ytest),epochs=100,batch_size=1,verbose=1)
```

11. Prédire les résultats sur les données de test :

```
test_predict=model.predict(X_test)
```

12. Calculer l’erreur quadratique moyenne :

```
from sklearn.metrics import mean_squared_error
import math
math.sqrt(mean_squared_error(ytest,test_predict))
```

13. Annuler la mise à l'échelle de ytest et le vecteur de prédictions :

```
test_predict=scaler.inverse_transform(test_predict)
ytest = scaler.inverse_transform(ytest)
```

PARTIE 3

Copyright - Tout droit réservé - OFPPT

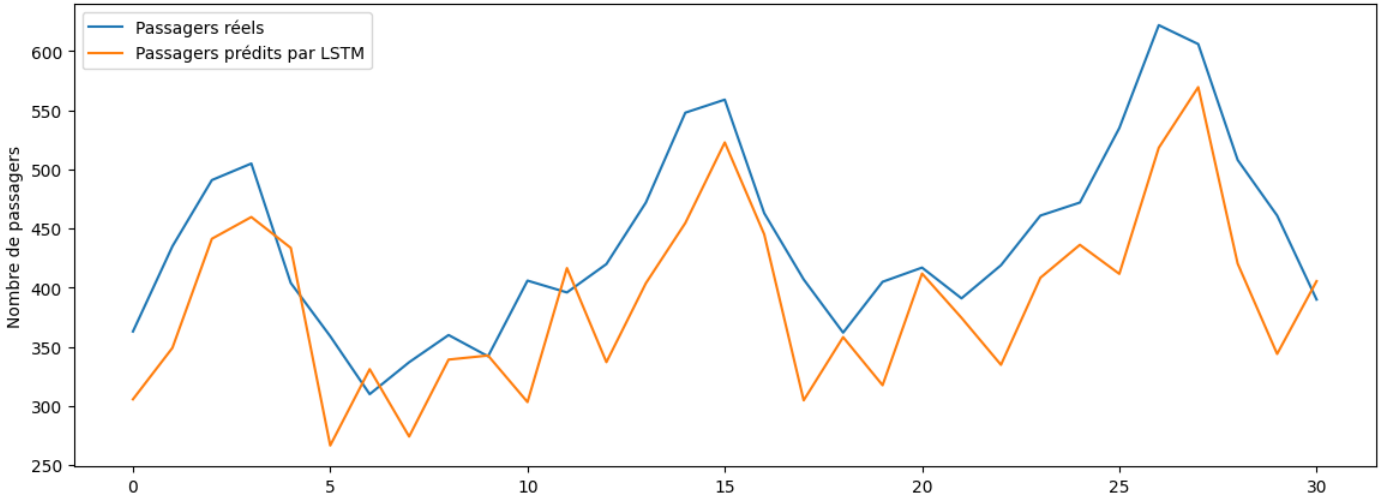
Activité 1

Correction

Exercice 1

14. Afficher la courbe des nombres de passagers prédits ainsi que les vraies valeurs des passagers :

```
plt.figure(figsize=(14,5))
plt.plot(ytest, label = 'real number of passengers')
plt.plot(test_predict, label = 'predicted number of passengers')
plt.ylabel('# passengers')
plt.legend()
plt.show()
```



PARTIE 3

Copyright - Tout droit réservé - OFPPT